

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH  
KHOA ĐIỆN - ĐIỆN TỬ  
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG

ĐỒ ÁN TỐT NGHIỆP

**TRIỂN KHAI, SO SÁNH VÀ ĐÁNH GIÁ CÁC THUẬT TOÁN MÃ HÓA ĐỐI  
XỨNG CHO HỆ THỐNG NHÚNG TÀI NGUYÊN HẠN CHẾ TRÊN NỀN  
TẢNG ESP32**

**NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ - TRUYỀN THÔNG**

Sinh viên: **LÊ TUẤN KHANH**  
MSSV: 22161267

TP. HỒ CHÍ MINH – 06/2026

**TRƯỜNG ĐẠI HỌC SƯ CÔNG NGHỆ KỸ THUẬT THÀNH PHỐ HỒ CHÍ MINH**  
**KHOA ĐIỆN - ĐIỆN TỬ**  
**BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG**

**ĐỒ ÁN TỐT NGHIỆP**

**TRIỂN KHAI, SO SÁNH VÀ ĐÁNH GIÁ CÁC THUẬT TOÁN MÃ HÓA ĐỐI  
XỨNG CHO HỆ THỐNG NHÚNG TÀI NGUYÊN HẠN CHẾ TRÊN NỀN  
TẢNG ESP32**

**NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ TRUYỀN THÔNG**

Sinh viên: **LÊ TUẤN KHANH**  
MSSV: 22161267

Hướng dẫn: **ThS. LÊ MINH**

TP. HỒ CHÍ MINH – 06/2026

## THÔNG TIN KHÓA LUẬN TỐT NGHIỆP

### 1. Thông tin sinh viên

Họ và tên sinh viên: Lê Tuấn Khanh

MSSV: 22161267

Email: khanhlthcm@gmail.com

Điện thoại: 0942359066

### 2. Thông tin đề tài

- Tên đề tài: TRIỂN KHAI, SO SÁNH VÀ ĐÁNH GIÁ CÁC THUẬT TOÁN MÃ HÓA ĐỐI XỨNG CHO HỆ THỐNG NHÚNG TÀI NGUYÊN HẠN CHẾ TRÊN NỀN TẢNG ESP32

- Đơn vị quản lý: Bộ môn Kỹ Thuật máy tính – Viễn Thông, Khoa Điện Điện Tử, Trường Đại Học Công Nghệ Kỹ Thuật Tp. Hồ Chí Minh.

- Thời gian thực hiện: Từ ngày 01 / 03 / 2026 đến ngày 20 / 06 / 2026

- Thời gian bảo vệ trước hội đồng: Ngày .... / .... / 20..

### 3. Lời cam đoan của sinh viên

Tôi Lê Tuấn Khanh cam đoan KLTN là công trình nghiên cứu của tôi, dưới sự hướng dẫn của Thạc sĩ Lê Minh. Kết quả công bố trong KLTN là trung thực và không sao chép từ bất kì công trình nào khác.

Tp. HCM, ngày ... tháng ... năm 2026

SV thực hiện đồ án

(Ký và ghi rõ họ tên)

.....

---

Giảng viên hướng dẫn xác nhận quyền báo cáo đã được chỉnh sửa theo đề nghị được ghi trong biên bản của Hội đồng đánh giá Khóa luận tốt nghiệp.

.....

Xác nhận của Bộ Môn

Tp. HCM, ngày ... tháng ... năm 2026

Giảng viên hướng dẫn

(Ký, ghi rõ họ tên và học hàm – học vị)

## BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: TRIỂN KHAI, SO SÁNH VÀ ĐÁNH GIÁ CÁC THUẬT TOÁN MÃ HÓA ĐỐI XỨNG CHO HỆ THỐNG NHÚNG TÀI NGUYÊN HẠN CHẾ TRÊN NỀN TẢNG ESP32 .....

Sinh viên: + Lê Tuấn Khanh..... MSSV: 22161267.....

Hướng dẫn: Thạc sĩ Lê Minh .....

---

### Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

*Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.*

.....

.....

2. Phương pháp thực hiện/ phân tích/ thiết kế:

*Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo*

.....

.....

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

*Phù hợp với mục tiêu đề tài; phân tích và đánh giá / kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.*

.....

4. Kết luận và đề xuất:

*Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.*

.....

5. Hình thức trình bày và bố cục báo cáo:

*Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.*

.....

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

*Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.*

.....  
7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

.....  
8. Đánh giá về sự trùng lặp của đề tài

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

.....  
9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa\* .....

.....  
.....  
.....  
.....

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

.....

**Đề nghị của giảng viên hướng dẫn**

Ghi rõ: “Bảo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp”

.....

*Tp. HCM, ngày ... tháng .... năm 2026*

**Người nhận xét**  
(Ký và ghi rõ họ tên)

## BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dùng cho giảng viên phản biện)

|            |                                                                                                                                |       |          |
|------------|--------------------------------------------------------------------------------------------------------------------------------|-------|----------|
| Đề tài:    | TRIỂN KHAI, SO SÁNH VÀ ĐÁNH GIÁ CÁC THUẬT TOÁN<br>MÃ HÓA ĐỐI XỨNG CHO HỆ THỐNG NHÚNG TÀI<br>NGUYÊN HẠN CHẾ TRÊN NỀN TẢNG ESP32 |       |          |
| Sinh viên: | Lê Tuấn Khanh                                                                                                                  | MSSV: | 22161267 |

### **Nhận xét bao gồm các nội dung sau đây:**

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và tính ứng dụng [15]:

*Đặt vấn đề rõ ràng, mục tiêu cụ thể [5]; đề tài có tính mới, cấp thiết [5]; đề tài có khả năng ứng dụng, tính sáng tạo [5].*

2. Phương pháp thực hiện/ phân tích/ thiết kế [25]:

*Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết [10]; có phân tích và đánh giá phù hợp [10]; có tính mới và tính sáng tạo [5]*

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế [25]:

*Phù hợp với mục tiêu [10]; phân tích và đánh giá / kiểm thử thiết kế hợp lý [10]; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy [5].*

4. Kết luận và đề xuất [10]:

*Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn [5]; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục [5].*

5. Hình thức trình bày, bố cục và chất lượng báo cáo [20]:

*Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu [5]; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt [15].*

6. Tài liệu trích dẫn [5]:

*Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.*

7. Đánh giá về sự trùng lặp của đề tài:

*Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.*

8. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa\*:

**Câu hỏi sinh viên phải trả lời trước hội đồng\*:**

---

|                  |                                            |                                                   |                   |             |
|------------------|--------------------------------------------|---------------------------------------------------|-------------------|-------------|
| <b>Kết luận:</b> | <b>Cho bảo vệ</b> <input type="checkbox"/> | <b>Không cho bảo vệ*</b> <input type="checkbox"/> | <b>Điểm tổng:</b> | <b>/100</b> |
|------------------|--------------------------------------------|---------------------------------------------------|-------------------|-------------|

---

*\*Trường hợp giáo viên phản biện không cho bảo vệ, vui lòng ghi rõ lý do:*

*Tp. HCM, ngày ... tháng .... năm 2026*

**Người phản biện**

## LỜI CẢM ƠN

Trong quá trình thực hiện đồ án tốt nghiệp với đề tài:

“Triển khai, so sánh và đánh giá các thuật toán mã hóa đối xứng cho hệ thống nhúng tài nguyên hạn chế trên nền tảng ESP32”, tác giả đã nhận được rất nhiều sự hỗ trợ, hướng dẫn và động viên từ quý thầy cô, gia đình và bạn bè.

Tác giả xin bày tỏ lòng biết ơn đến Thạc sĩ Lê Minh, người đã trực tiếp hướng dẫn, tận tình hỗ trợ trong suốt quá trình thực hiện đồ án. Những góp ý chuyên môn, kinh nghiệm nghiên cứu của giảng viên hướng dẫn là nền tảng quan trọng giúp tác giả hoàn thành đề tài này.

Bên cạnh đó, tác giả cũng xin cảm ơn quý thầy cô trong Khoa Điện – Điện tử đã truyền đạt những kiến thức quý báu trong suốt quá trình học tập tại trường. Đây là nền tảng quan trọng giúp tác giả có đủ kiến thức và kỹ năng để thực hiện đề tài tốt nghiệp.

Tác giả cũng xin gửi lời cảm ơn đến gia đình đã luôn hỗ trợ tinh thần hết mình trong quá trình làm đồ án.

Xin chân thành cảm ơn!



## TÓM TẮT

Trong những năm gần đây, sự phát triển mạnh mẽ của Internet of Things (IoT) và các hệ thống nhúng đã đặt ra yêu cầu ngày càng cao về khả năng bảo mật dữ liệu trên các thiết bị có tài nguyên hạn chế. Trong bối cảnh đó, việc lựa chọn thuật toán mã hóa phù hợp cần phải đảm bảo đồng thời nhiều yếu tố như độ an toàn, hiệu năng xử lý, mức sử dụng tài nguyên và khả năng đảm bảo toàn vẹn dữ liệu.

Đề án tập trung vào việc xây dựng hệ thống thực nghiệm để so sánh và đánh giá ba thuật toán mã hóa đối xứng gồm AES, ChaCha20 và Ascon trên vi điều khiển ESP32. Trong đó, AES đại diện cho nhóm thuật toán mã hóa khối truyền thống, ChaCha20 đại diện cho nhóm mã hóa dòng tối ưu cho phần mềm, còn Ascon là thuật toán mật mã hạng nhẹ hiện đại được chuẩn hóa bởi NIST dành cho các hệ thống tài nguyên hạn chế.

Hệ thống thực nghiệm được xây dựng dựa trên mô hình gồm hai nút ESP32 giao tiếp với nhau thông qua UART, sử dụng cảm biến DHT22 để thu thập dữ liệu nhiệt độ, độ ẩm từ môi trường. Đề tài xây dựng ba kịch bản thử nghiệm bao gồm: truyền dữ liệu định kỳ, truyền dữ liệu liên tục và truyền dữ liệu trong điều kiện có lỗi. Các kịch bản này cho phép đánh giá thuật toán trong nhiều điều kiện hoạt động khác nhau, từ tải thấp đến tải cao và môi trường truyền có nhiễu.

Trong quá trình thực nghiệm, các thông số như thời gian mã hóa, thời gian giải mã, độ trễ toàn hệ thống, thông lượng, dung lượng Flash, RAM và khả năng phát hiện lỗi được thu thập và phân tích trên nền tảng ESP32 NodeMCU thực tế. Kết quả thực nghiệm cho thấy ChaCha20 là thuật toán có hiệu năng hoạt động tốt nhất trong ba thuật toán khảo sát, với độ trễ thấp, khả năng hoạt động ổn định và mức sử dụng tài nguyên hệ thống thấp, phù hợp với các hệ thống truyền dữ liệu định kỳ và truyền liên tục. Trong khi đó, Ascon cho thấy ưu điểm nổi bật về độ tin cậy và khả năng đảm bảo toàn vẹn dữ liệu nhờ cơ chế xác thực tích hợp, phù hợp với các ứng dụng yêu cầu mức độ bảo mật và độ tin cậy cao. AES mặc dù không nổi bật về hiệu năng hay khả năng xác thực dữ liệu, nhưng vẫn duy trì khả năng hoạt động ổn định và có ưu thế về tính phổ biến cũng như khả năng tương thích trong nhiều hệ thống hiện nay.

Thông qua việc thực hiện trên phần cứng ESP32 và đánh giá trong nhiều kịch bản thực tế, đề tài cung cấp cái nhìn toàn diện về hiệu năng và tính phù hợp của các thuật toán mã hóa đối xứng trong hệ thống nhúng. Kết quả của đề tài có thể được sử dụng làm tài liệu tham khảo cho các nghiên cứu và ứng dụng liên quan đến bảo mật dữ liệu trong IoT và hệ thống nhúng tài nguyên hạn chế.

# MỤC LỤC

|                                              |             |
|----------------------------------------------|-------------|
| <b>DANH MỤC HÌNH .....</b>                   | <b>XII</b>  |
| <b>DANH MỤC BẢNG .....</b>                   | <b>XIII</b> |
| <b>CÁC TỪ VIẾT TẮT .....</b>                 | <b>XIV</b>  |
| <b>CHƯƠNG 1 GIỚI THIỆU .....</b>             | <b>1</b>    |
| 1.1 GIỚI THIỆU .....                         | 1           |
| 1.2 MỤC TIÊU ĐỀ TÀI .....                    | 2           |
| 1.3 PHẠM VI GIỚI HẠN ĐỀ TÀI .....            | 3           |
| 1.4 PHƯƠNG PHÁP NGHIÊN CỨU .....             | 3           |
| 1.5 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU .....    | 4           |
| 1.6 BỐ CỤC QUYỀN BẢO CÁO .....               | 4           |
| <b>CHƯƠNG 2 CƠ SỞ LÝ THUYẾT .....</b>        | <b>5</b>    |
| 2.1 THUẬT TOÁN AES .....                     | 5           |
| 2.1.1 <i>Tổng quan</i> .....                 | 5           |
| 2.1.2 <i>Cấu trúc thuật toán</i> .....       | 5           |
| 2.1.3 <i>Nguyên lý hoạt động</i> .....       | 6           |
| 2.1.4 <i>Đánh giá</i> .....                  | 7           |
| 2.2 THUẬT TOÁN CHACHA20 .....                | 7           |
| 2.2.1 <i>Tổng quan</i> .....                 | 7           |
| 2.2.2 <i>Cấu trúc thuật toán</i> .....       | 8           |
| 2.2.3 <i>Nguyên lý hoạt động</i> .....       | 8           |
| 2.2.4 <i>Đánh giá</i> .....                  | 9           |
| 2.3 THUẬT TOÁN ASCON .....                   | 9           |
| 2.3.1 <i>Tổng quan</i> .....                 | 9           |
| 2.3.2 <i>Cấu trúc thuật toán Ascon</i> ..... | 10          |
| 2.3.3 <i>Nguyên lý hoạt động</i> .....       | 11          |
| 2.3.4 <i>Đánh giá</i> .....                  | 13          |
| 2.4 GIAO TIẾP UART .....                     | 13          |

|                 |                                                                                                                                          |           |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>CHƯƠNG 3</b> | <b>THIẾT KẾ HỆ THỐNG TRIỂN KHAI, SO SÁNH VÀ ĐÁNH GIÁ CÁC THUẬT TOÁN ĐỐI XỨNG .....</b>                                                   | <b>15</b> |
| 3.1             | YÊU CẦU CỦA HỆ THỐNG .....                                                                                                               | 15        |
| 3.2             | MÔ HÌNH HỆ THỐNG .....                                                                                                                   | 16        |
| 3.3             | THIẾT KẾ PHẦN CỨNG .....                                                                                                                 | 17        |
| 3.3.1           | <i>Chức năng của phần cứng.....</i>                                                                                                      | <i>17</i> |
| 3.3.2           | <i>Sơ đồ khối phần cứng.....</i>                                                                                                         | <i>17</i> |
| 3.3.3           | <i>Thiết kế từng khối.....</i>                                                                                                           | <i>18</i> |
| 3.3.3.1         | <i>Khối cảm biến.....</i>                                                                                                                | <i>18</i> |
| 3.3.3.2         | <i>Khối hiển thị.....</i>                                                                                                                | <i>19</i> |
| 3.3.3.3         | <i>Khối xử lý trung tâm .....</i>                                                                                                        | <i>21</i> |
| 3.3.3.4         | <i>Sơ đồ nguyên lý toàn hệ thống.....</i>                                                                                                | <i>24</i> |
| 3.4             | THIẾT KẾ KỊCH BẢN SO SÁNH VÀ ĐÁNH GIÁ THUẬT TOÁN.....                                                                                    | 25        |
| 3.4.1           | <i>Thiết kế kịch bản thử nghiệm số 1: Nút cảm biến truyền dữ liệu định kỳ</i>                                                            | <i>25</i> |
| 3.4.2           | <i>Thiết kế kịch bản thử nghiệm số 2: Truyền dữ liệu liên tục .....</i>                                                                  | <i>28</i> |
| 3.4.3           | <i>Thiết kế kịch bản thử nghiệm số 3: Đánh giá độ tin cậy và toàn vẹn dữ liệu của các thuật toán mã hóa trong điều kiện có lỗi .....</i> | <i>32</i> |
| <b>CHƯƠNG 4</b> | <b>KẾT QUẢ.....</b>                                                                                                                      | <b>37</b> |
| 4.1             | MÔ HÌNH PHẦN CỨNG THỰC NGHIỆM.....                                                                                                       | 37        |
| 4.2             | KẾT QUẢ THỰC THI HỆ THỐNG .....                                                                                                          | 37        |
| 4.3             | KẾT QUẢ ĐÁNH GIÁ HỆ THỐNG.....                                                                                                           | 41        |
| 4.3.1           | <i>Kịch bản thử nghiệm số 1: Nút cảm biến truyền dữ liệu định kỳ.....</i>                                                                | <i>41</i> |
| 4.3.2           | <i>Kịch bản thử nghiệm số 2: Truyền dữ liệu liên tục .....</i>                                                                           | <i>43</i> |
| 4.3.3           | <i>Kịch bản thử nghiệm số 3: Đánh giá độ tin cậy và toàn vẹn dữ liệu của các thuật toán mã hóa trong điều kiện có lỗi .....</i>          | <i>46</i> |
| <b>CHƯƠNG 5</b> | <b>KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN .....</b>                                                                                                | <b>49</b> |
| 5.1             | KẾT LUẬN .....                                                                                                                           | 49        |
| 5.2             | HƯỚNG PHÁT TRIỂN.....                                                                                                                    | 51        |

|                                 |           |
|---------------------------------|-----------|
| <b>PHỤ LỤC .....</b>            | <b>52</b> |
| <b>TÀI LIỆU THAM KHẢO .....</b> | <b>53</b> |

## DANH MỤC HÌNH

|                                                                                               |    |
|-----------------------------------------------------------------------------------------------|----|
| Hình 2.1 Quy trình mã hóa của thuật toán AES .....                                            | 6  |
| Hình 2.2 Quy trình giải mã của thuật toán AES .....                                           | 6  |
| Hình 2.3 Bảng trạng thái nội bộ của thuật toán ChaCha20 .....                                 | 8  |
| Hình 2.4 Lưu đồ thuật toán ChaCha20 .....                                                     | 9  |
| Hình 2.5 Quy trình mã hóa thuật toán Ascon [4].....                                           | 12 |
| Hình 2.6 Quy trình giải mã thuật toán Ascon [4] .....                                         | 12 |
| Hình 3.1 Mô hình hệ thống .....                                                               | 16 |
| Hình 3.2 Sơ đồ khối phần cứng của hệ thống thực nghiệm.....                                   | 17 |
| Hình 3.3 Cảm biến DHT22 [11] .....                                                            | 19 |
| Hình 3.4 Sơ đồ nguyên lí của cảm biến DHT22.....                                              | 19 |
| Hình 3.5 Màn hình OLED SSD1306 [21] .....                                                     | 21 |
| Hình 3.6 Sơ đồ nguyên lí của màn hình OLED SSD1306 .....                                      | 21 |
| Hình 3.7 Lưu đồ hoạt động của ESP32 phía gửi và phía nhận.....                                | 23 |
| Hình 3.8 Sơ đồ chân của ESP32 NodeMCU .....                                                   | 24 |
| Hình 3.9 Sơ đồ nguyên lý của khối xử lý trung tâm của nút gửi và nút nhận .....               | 24 |
| Hình 3.10 Mạch sơ đồ nguyên lí.....                                                           | 25 |
| Hình 3.11 Lưu đồ giải thuật của kịch bản 1 .....                                              | 27 |
| Hình 3.12 Lưu đồ giải thuật của kịch bản 2.....                                               | 32 |
| Hình 3.13 Lưu đồ giải thuật kịch bản 3 .....                                                  | 35 |
| Hình 4.1 Mô hình phần cứng thực nghiệm của hệ thống .....                                     | 37 |
| Hình 4.2 Quá trình đo đạc trên phần cứng và Serial Monitor của kịch bản thử nghiệm số 1 ..... | 38 |
| Hình 4.3 Kết quả hoạt động của kịch bản thử nghiệm số 1 .....                                 | 38 |
| Hình 4.4 Quá trình hoạt động của kịch bản thử nghiệm số 2 .....                               | 39 |
| Hình 4.5 Kết quả hoạt động của kịch bản thử nghiệm số 2 .....                                 | 39 |
| Hình 4.6 Quá trình hoạt động của kịch bản số 3.....                                           | 40 |
| Hình 4.7 Kết quả hoạt động của kịch bản số 3 .....                                            | 40 |

## DANH MỤC BẢNG

|                                                                                |    |
|--------------------------------------------------------------------------------|----|
| <b>Bảng 3.1</b> So sánh các linh kiện cảm biến .....                           | 18 |
| <b>Bảng 3.2</b> So sánh các màn hình.....                                      | 20 |
| <b>Bảng 3.3</b> So sánh các vi điều khiển.....                                 | 22 |
| <b>Bảng 4.1</b> Hiệu năng hoạt động của các thuật toán của kịch bản số 1 ..... | 41 |
| <b>Bảng 4.2</b> Hiệu suất hoạt động của hệ thống của kịch bản số 1 .....       | 42 |
| <b>Bảng 4.3</b> Kết quả tài nguyên sử dụng của kịch bản số 1 .....             | 43 |
| <b>Bảng 4.4</b> Hiệu năng hoạt động của các thuật toán của kịch bản số 2 ..... | 44 |
| <b>Bảng 4.2</b> Hiệu suất hoạt động của hệ thống của kịch bản số 2 .....       | 45 |
| <b>Bảng 4.6</b> Kết quả tài nguyên sử dụng của kịch bản số 2.....              | 46 |
| <b>Bảng 4.7</b> Mức độ lỗi của gói dữ liệu bị lỗi sau giải mã .....            | 47 |
| <b>Bảng 4.8</b> Kết quả đánh giá độ tin cậy và toàn vẹn .....                  | 48 |

## CÁC TỪ VIẾT TẮT

| <b>Từ viết tắt</b> | <b>Tiếng Anh</b>                               | <b>Ý nghĩa</b>                               |
|--------------------|------------------------------------------------|----------------------------------------------|
| AEAD               | Authenticated Encryption with Associated Data  | Mã hóa kèm xác thực dữ liệu                  |
| AES                | Advanced Encryption Standard                   | Chuẩn mã hóa nâng cao                        |
| Bps                | Bit per second                                 | Tốc độ bit mỗi giây                          |
| CBC                | Cipher Block Chaining                          | Chế độ mã khối liên kết                      |
| DHT22              | Digital Humidity and Temperature Sensor 22     | Cảm biến nhiệt độ và độ ẩm                   |
| ECC                | Elliptic Curve Cryptography                    | Mật mã đường cong ellipticSPI                |
| ESP32              | Espressif 32                                   | Vi điều khiển ESP32                          |
| GDDRAM             | Graphic Display Data RAM                       | Bộ nhớ đồ họa hiển thị                       |
| GPIO               | General Purpose Input Output                   | Chân vào ra đa dụng                          |
| I2C                | Inter-Integrated Circuit                       | Chuẩn giao tiếp nối tiếp I2C                 |
| IoT                | Internet of Things                             | Internet vạn vật                             |
| IV                 | Initialization Vector                          | Vector khởi tạo                              |
| KB/s               | Kilobytes per second                           | Kilobyte trên giây                           |
| MB/s               | Megabytes per second                           | Megabyte trên giây                           |
| MQTT               | Message Queuing Telemetry Transport            | Giao thức truyền tin IoT                     |
| NIST               | National Institute of Standards and Technology | Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ |
| NC                 | Not connected                                  | Chân không sử dụng                           |
| OLED               | Organic Light-Emitting Diode                   | Điốt phát quang hữu cơ                       |
| RAM                | Random Access Memory                           | Bộ nhớ truy cập ngẫu nhiên                   |
| RFC                | Request for Comments                           | Tài liệu tiêu chuẩn kỹ thuật Internet        |
| RH                 | Relative Humidity                              | Độ ẩm tương đối                              |



|        |                                                   |                                         |
|--------|---------------------------------------------------|-----------------------------------------|
| RSA    | Rivest–Shamir–Adleman                             | Thuật toán mật mã bất đối xứng          |
| S-box  | Substitution Box                                  | Hộp thay thế                            |
| SPI    | Serial Peripheral Interface                       | Chuẩn giao tiếp ngoại vi nối tiếp       |
| SPN    | Substitution Permutation Network                  | Mạng thay thế – hoán vị                 |
| TCP/IP | Transmission Control Protocol / Internet Protocol | Bộ giao thức mạng Internet              |
| TFT    | Thin-Film Transistor                              | Công nghệ màn hình transistor màng mỏng |
| UART   | Universal Asynchronous Receiver Transmitter       | Chuẩn truyền thông nối tiếp bất đồng bộ |
| USB    | Universal Serial Bus                              | Chuẩn kết nối USB                       |
| XOR    | Exclusive OR                                      | Phép toán XOR                           |

# CHƯƠNG 1

## GIỚI THIỆU

### 1.1 GIỚI THIỆU

Trong những năm gần đây, sự phát triển của Internet of Things (IoT) và hệ thống nhúng đã thúc đẩy nhiều ứng dụng nhà thông minh, y tế, nông nghiệp và công nghiệp 4.0. Các thiết bị trong các hệ thống này thường có tài nguyên hạn chế về bộ nhớ, năng lượng và khả năng xử lý, nhưng vẫn phải đảm bảo tính toàn vẹn và bảo mật của dữ liệu. Vì vậy, việc nghiên cứu các thuật toán mã hóa cho phù hợp với hệ thống nhúng tài nguyên hạn chế là một hướng nghiên cứu cần thiết.

Hiện nay, các thuật toán mã hóa đối xứng như AES được sử dụng rộng rãi nhờ độ an toàn và đã được chuẩn hóa bởi NIST [1], [18], [19]. Tuy nhiên, AES thường yêu cầu tài nguyên xử lý tương đối lớn khi triển khai trên các thiết bị nhúng không có hỗ trợ phần cứng. Để cải thiện hiệu năng, các thuật toán mã hóa nhẹ như ChaCha20 [3] và Ascon [4] đã được đề xuất để tối ưu cho môi trường tài nguyên hạn chế. Nhiều nghiên cứu trước đây cho thấy ChaCha20 có ưu thế về tốc độ xử lý phần mềm, trong khi Ascon hỗ trợ cơ chế mã hóa kèm xác thực dữ liệu và tối ưu mức tài nguyên sử dụng.

Tuy nhiên, các nghiên cứu trước chủ yếu đánh giá các thuật toán riêng lẻ hoặc chỉ khảo sát trong một số điều kiện thử nghiệm hạn chế. Một số nghiên cứu chỉ dừng ở mức mô phỏng hoặc chưa triển khai các thuật toán trên cùng một nền tảng phần cứng để đảm bảo tính khách quan khi so sánh. Ngoài ra, các yếu tố như thông lượng hệ thống, mức sử dụng tài nguyên bộ nhớ và khả năng đảm bảo tính toàn vẹn dữ liệu chưa được phân tích đồng thời trong nhiều kịch bản truyền dữ liệu khác nhau.

Xuất phát từ điều đó, đề tài “Triển khai, so sánh và đánh giá các thuật toán mã hóa đối xứng cho hệ thống nhúng tài nguyên hạn chế trên nền tảng ESP32” được thực hiện nhằm triển khai và đánh giá các thuật toán AES, ChaCha20 và Ascon trên nền tảng ESP32. Đề tài tập trung xây dựng các kịch bản truyền dữ liệu thực tế để thu thập và so sánh các thông số như thời gian xử lý, thông lượng, tài nguyên và khả năng xử lý lỗi của thuật toán.

Đề tài sử dụng ESP32 làm nền tảng thí nghiệm, xây dựng chương trình mã hóa và giải mã cho từng thuật toán, thiết kế các kịch bản kiểm thử nhằm đánh giá hiệu năng, độ tin cậy của hệ thống trong cùng một điều kiện thử nghiệm.

So với các nghiên cứu trước, đề tài có ưu điểm là triển khai trực tiếp trên phần cứng thực thay vì mô phỏng, đồng thời đánh giá nhiều tiêu chí trong cùng một môi trường thực nghiệm để đảm bảo tính khách quan khi so sánh. Kết quả nghiên cứu dự kiến sẽ cung cấp số liệu thực nghiệm cụ thể về hiệu năng của từng thuật toán, từ đó đưa ra nhận xét và khuyến nghị lựa chọn thuật toán phù hợp cho các ứng dụng hệ thống nhúng tài nguyên hạn chế.

## **1.2 MỤC TIÊU ĐỀ TÀI**

Mục tiêu chính của đề tài là xây dựng hệ thống thực nghiệm để so sánh và đánh giá hiệu năng của các thuật toán mã hóa đối xứng trên hệ thống nhúng tài nguyên hạn chế, cụ thể:

Sử dụng các thuật toán mã hóa đối xứng phổ biến trên vi điều khiển ESP32, bao gồm: AES, ChaCha20, Ascon.

Xây dựng các kịch bản thực nghiệm nhằm đánh giá hiệu năng của các thuật toán trong các kiểu hoạt động khác nhau như truyền định kỳ, truyền liên tục, hoạt động trong điều kiện có lỗi.

Đánh giá các tiêu chí sau:

- Thời gian mã hóa và giải mã.
- Thời gian truyền và độ trễ toàn hệ thống.
- Thông lượng thuật toán, thông lượng hệ thống và thông lượng thực nghiệm.

- Mức sử dụng tài nguyên phần cứng (RAM, Flash).
- Khả năng phát hiện lỗi và đảm bảo tính toàn vẹn dữ liệu.
- Độ ổn định và hiệu năng của các thuật toán trong điều kiện tải khác nhau.

So sánh và đánh giá sự phù hợp của các thuật toán trong từng kịch bản ứng dụng cụ thể.

### **1.3 PHẠM VI GIỚI HẠN ĐỀ TÀI**

Đề tài tập trung vào việc đánh giá các thuật toán mã hóa đối xứng trong môi trường hệ thống nhúng với các giới hạn sau:

- Chỉ xét các thuật toán mã hóa đối xứng, không bao gồm các loại thuật toán bất đối xứng (RSA, ECC).
- Việc triển khai và thực nghiệm trên nền tảng ESP32 nên kết quả mang tính đặc thù cho nền tảng này và không đại diện cho tất cả hệ thống nhúng.
- Các thuật toán được sử dụng ở mức thư viện có sẵn hoặc triển khai chuẩn, không thực hiện tối ưu hóa ở mức phần cứng.
- Hệ thống truyền thông dùng giao tiếp UART, không xét đến các giao thức mạng phức tạp như TCP/IP hoặc MQTT.
- Việc đánh giá bảo mật chủ yếu dựa trên đặc tính của thuật toán và khả năng xác thực tính toàn vẹn dữ liệu, chưa đi sâu vào tấn công nâng cao như tấn công cạnh kênh (side-channel attacks).

### **1.4 PHƯƠNG PHÁP NGHIÊN CỨU**

Trong quá trình thực hiện đề tài, tác giả thực hiện sẽ sử dụng các phương pháp nghiên cứu sau:

- Nghiên cứu lý thuyết hoạt động của các thuật toán AES, ChaCha20 và Ascon.

- Triển khai các thuật toán trên ESP32 được thiết kế theo các kịch bản kiểm thử thực tế, thu thập dữ liệu đo đạc như thời gian xử lý, thông lượng, bộ nhớ sử dụng.
- So sánh và đánh giá các thuật toán trong cùng một điều kiện thử nghiệm, phân tích và đưa ra nhận xét khách quan.
- Sử dụng dữ liệu thực nghiệm để tính toán các thông số hiệu năng, trực quan hóa và diễn giải kết quả.

## **1.5 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU**

Đối tượng nghiên cứu của đề tài là các thuật toán mã hóa đối xứng AES, ChaCha20 và Ascon. Việc đánh giá được thực hiện trên nền tảng vi điều khiển ESP32 nhằm đo lường, so sánh và đánh giá các tiêu chí hiệu năng, mức sử dụng tài nguyên, bảo mật trong môi trường tài nguyên hạn chế.

Đề tài sẽ tập trung nghiên cứu và đánh giá hiệu năng của các thuật toán trong các ứng dụng IoT và hệ thống nhúng tài nguyên hạn chế. Việc đánh giá hiệu năng được thực hiện thông qua các kịch bản thử nghiệm cụ thể, không mở rộng sang hệ thống server hoặc cloud. Nghiên cứu chỉ xem xét hiệu quả thực tế của thuật toán trong việc truyền dữ liệu giữa các nút nhúng.

## **1.6 BỐ CỤC QUYỀN BÁO CÁO**

Nội dung báo cáo bao gồm các chương như sau:

- Chương 1: Giới thiệu: Trình bày tổng quan về đề tài, mục tiêu, phạm vi, phương pháp nghiên cứu và cấu trúc báo cáo.
- Chương 2: Cơ sở lý thuyết: Giới thiệu chi tiết về các thuật toán mã hóa đối xứng (AES, ChaCha20, Ascon)
- Chương 3: Thiết kế hệ thống và kịch bản thực nghiệm: Mô tả kiến trúc hệ thống, nền tảng ESP32 và các kịch bản kiểm thử.
- Chương 4: Kết quả thực nghiệm và đánh giá: Trình bày kết quả đo đạc, phân tích và so sánh hiệu năng giữa các thuật toán.
- Chương 5: Kết luận và hướng phát triển: Tổng kết kết quả đạt được, đưa ra nhận xét và đề xuất hướng nghiên cứu trong tương lai.

# CHƯƠNG 2

## CƠ SỞ LÝ THUYẾT

### 2.1 THUẬT TOÁN AES

#### 2.1.1 Tổng quan

AES (Advanced Encryption Standard) là thuật toán mã hóa đối xứng dạng khối được chuẩn hóa bởi Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST) trong tiêu chuẩn FIPS PUB 197 vào năm 2001 [1].

AES hoạt động với khối dữ liệu đầu vào có kích thước cố định 128 bit, với các độ dài khóa 128 bit, 192 bit và 256 bit [1]. Trong phạm vi đề tài, phiên bản AES-128 được sử dụng do có mức cân bằng tốt giữa độ bảo mật, chi phí tính toán, phù hợp với hệ thống nhúng tài nguyên hạn chế.

Trong quá trình triển khai thí nghiệm, thuật toán AES-128 được sử dụng kết hợp với chế độ hoạt động CBC để tăng tính bảo mật cho dữ liệu truyền. Chế độ CBC giúp tăng cường tính bảo mật so với ECB bằng cách tạo sự phụ thuộc giữa các khối dữ liệu thông qua phép XOR với khối trước đó, từ đó hạn chế việc xuất hiện các mẫu lặp trong bản mã [2].

#### 2.1.2 Cấu trúc thuật toán

AES được xây dựng dựa trên cấu trúc mạng thay thế - hoán vị (SPN) [1]. Dữ liệu đầu vào được biểu hiện dưới dạng bảng ma trận trạng thái kích thước 4x4 bytes. Quá trình mã hóa AES bao gồm nhiều vòng lặp, phụ thuộc vào độ dài khóa: 10 vòng đối với AES-128, 12 vòng với AES-192, 14 vòng đối với AES-256 [1].

Mỗi vòng mã hóa bao gồm các bước hoạt động sau:

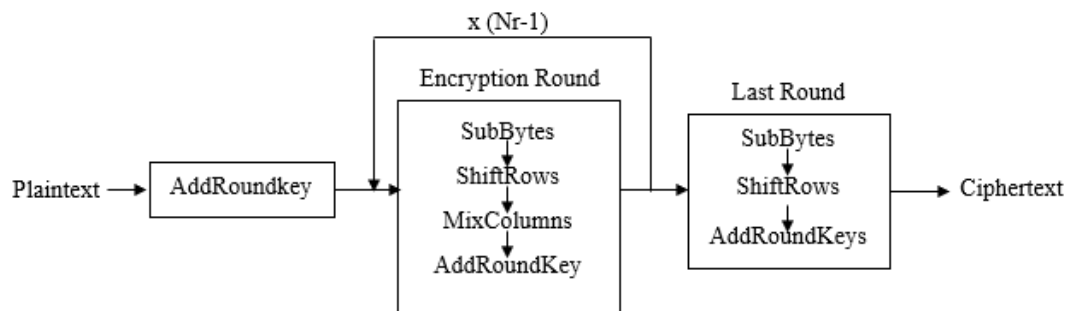
- SubBytes: Thay thế từng byte của trạng thái bằng giá trị tương ứng trong bản S-box nhằm tăng tính phi tuyến.

- ShiftRows: Dịch vòng các hàng của ma trận trạng thái để tăng khả năng khuếch tán.
- MixColumns: Thực hiện phép biến đổi tuyến tính trên từng cột nhằm tăng cường sự phụ thuộc giữa các byte.
- AddRoundKey: Thực hiện phép XOR giữa trạng thái và khóa vòng.

Vòng cuối cùng không bao gồm bước MixColumns. Quy trình giải mã thực hiện ngược lại các bước tương ứng [1].

### 2.1.3 Nguyên lý hoạt động

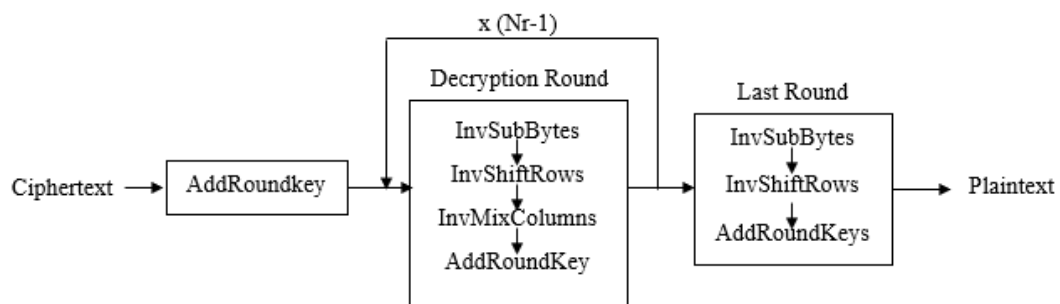
Quy trình mã hóa AES được minh họa trong hình 2.1.1, trong đó dữ liệu đầu vào (plaintext) được xử lý qua nhiều vòng biến đổi để tạo ra bản mã (ciphertext).



Hình 2.1 Quy trình mã hóa của thuật toán AES

Cụ thể, bước SubBytes tạo ra tính phi tuyến, bước ShiftRows và MixColumns làm khuếch tán dữ liệu trên toàn bộ ma trận trạng thái.

Quá trình giải mã được thực hiện thông qua các phép biến đổi ngược tương ứng, như hình minh họa 2.1.2. Các bước InvSubBytes, InvShiftRows và InvColumns sẽ có trạng thái hoạt động ngược với SubBytes, ShiftRows và Columns ở bước mã hóa.



Hình 2.2 Quy trình giải mã của thuật toán AES

Quy trình mã hóa và giải mã được thực hiện  $N_r$  vòng, với  $N_r$  là số vòng mã hóa và giải mã tương ứng với độ dài khóa.

Đối với chế độ CBC, mỗi khối đầu vào trước khi mã hóa sẽ được thực hiện phép XOR với khối kết quả của vòng mã hóa trước đó, giúp tăng tính liên kết giữa các khối dữ liệu và hạn chế các mẫu lặp lại trong bản mã [2].

#### **2.1.4 Đánh giá**

AES là một trong những thuật toán mã hóa đối xứng có độ bảo mật cao và được sử dụng rộng rãi trong nhiều hệ thống thực tế. Nhờ cấu trúc SPN và các phép biến đổi lặp lại qua nhiều vòng mã hóa. AES có khả năng chống lại nhiều dạng tấn công mật mã hiện đại.

Tuy nhiên, do yêu cầu phép tính phức tạp và việc sử dụng bảng tra cứu S-box, nên AES tiêu tốn đáng kể tài nguyên tính toán và bộ nhớ khi triển khai trên các hệ thống nhúng tài nguyên hạn chế. Điều này có thể ảnh hưởng đến hiệu năng và mức tiêu thụ năng lượng của hệ thống.

Trong đề tài, AES được sử dụng làm thuật toán tham chiếu để so sánh hiệu năng với các phương pháp mã hóa khác trên ESP32. Chế độ CBC hỗ trợ đánh giá thời gian xử lý và throughput trong mô hình truyền dữ liệu theo khối.

Ngoài ra, đặc điểm phụ thuộc giữa các khối dữ liệu trong CBC cũng tạo điều kiện để quan sát ảnh hưởng của cơ chế mã hóa đến độ trễ truyền trong các kịch bản truyền liên tục, đóng vai trò làm cơ sở so sánh với các thuật toán xử lý dạng dòng hoặc tích hợp xác thực.

## **2.2 THUẬT TOÁN CHACHA20**

### **2.2.1 Tổng quan**

ChaCha20 là thuật toán mã hóa dòng được thiết kế bởi Daniel J. Bernstein, là phiên bản cải tiến của Salsa20 và được chuẩn hóa trong RFC 8439 [3]. Thuật toán này được thiết kế nhằm đạt hiệu năng cao trên phần mềm, đặc biệt trong các hệ thống không có phần cứng hỗ trợ tăng tốc như AES.



ChaCha20 sử dụng khóa có độ dài 256 bits, kết hợp với nonce 96 bits và bộ đếm counter 32 bits để tạo chuỗi khóa. Chuỗi khóa này sau đó được XOR với dữ liệu đầu vào để tạo ra bản mã hóa.

### 2.2.2 Cấu trúc thuật toán

ChaCha20 sử dụng một bảng trạng thái nội bộ dưới dạng ma trận 4x4, mỗi phần tử có độ dài 32 bit. Trạng thái này bao gồm 4 hằng số cố định ở hàng 1, 8 phần tử khóa ở hàng 2 và 3, bộ đếm counter ở hàng 4 cột một và 3 phần tử nonce ở 3 vị trí còn lại ở hàng 4 [3].

|   |   |   |   |
|---|---|---|---|
| C | C | C | C |
| K | K | K | K |
| K | K | K | K |
| B | N | N | N |

C: Hằng số

K: Phần tử khóa

B: Bộ đếm counter

N: Nonce

Hình 2.3 Bảng trạng thái nội bộ của thuật toán ChaCha20

Quá trình sinh chuỗi khóa được thực hiện thông qua 20 vòng biến đổi, tương đương 10 double rounds. Mỗi double round bao gồm hai bước: column rounds và diagonal rounds [3].

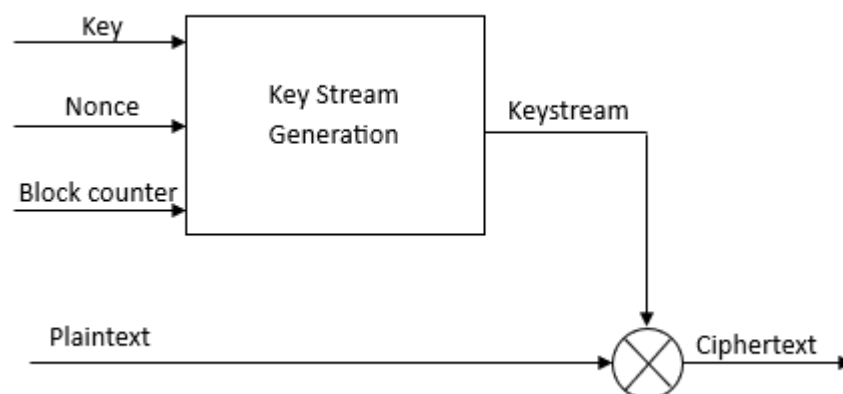
Các phép toán cơ bản được sử dụng trong ChaCha20 bao gồm:

- Phép cộng module  $2^{32}$
- Phép XOR
- Phép quay vòng bit (rotation)

Chỉ nhờ sử dụng các phép toán cơ bản, ChaCha20 có thể đạt hiệu năng cao trên các nền tảng vi điều khiển [3].

### 2.2.3 Nguyên lý hoạt động

Nguyên lý hoạt động của ChaCha20 được minh họa trong hình 2.2.1, trong đó trạng thái ban đầu được khởi tạo từ khóa, nonce và counter. Sau đó, thuật toán thực hiện 20 vòng biến đổi để tạo một khối keystream.



Hình 2.4 Lưu đồ thuật toán ChaCha20

Keystream sau khi sinh ra được XOR với dữ liệu đầu vào để tạo thành bản mã. Quá trình giải mã được thực hiện tương tự bằng cách XOR bản mã với cùng keystream, do tính chất đối xứng của XOR.

#### 2.2.4 Đánh giá

ChaCha20 có nhiều ưu điểm nổi bật khi triển khai trên các hệ thống nhúng tài nguyên hạn chế. Nhờ sử dụng các phép toán đơn giản và không yêu cầu bảng tra cứu, thuật toán có thể đạt hiệu năng cao và tiêu thụ ít tài nguyên.

Nhờ không sử dụng bảng tra cứu như AES, ChaCha20 giảm đáng kể nguy cơ rò rỉ thông tin qua kênh bên và phù hợp hơn với các vi điều khiển không có bộ nhớ cache hoặc có tài nguyên hạn chế.

Tuy nhiên, ChaCha20 là thuật toán mã hóa dòng và không tích hợp cơ chế xác thực dữ liệu. Do đó, để đảm bảo tính toàn vẹn và xác thực, thuật toán thường được kết hợp với Poly1305 tạo thành ChaCha20-Poly1305 [3].

Trong hệ thống thực nghiệm, ChaCha20 đại diện cho nhóm mã hóa dòng, phù hợp với các kịch bản truyền dữ liệu liên tục với kích thước biến đổi, đặc biệt phù hợp với đặc trưng của các ứng dụng IoT.

### 2.3 THUẬT TOÁN ASCON

#### 2.3.1 Tổng quan

Ascon là một họ thuật toán mật mã hạng nhẹ được thiết kế nhằm đáp ứng nhu cầu bảo mật cho các hệ thống nhúng có tài nguyên hạn chế như Internet of

Things (IoT), hệ thống nhúng và các thiết bị tiêu thụ năng lượng thấp [4], [8]. Năm 2023, Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST) đã chính thức chọn Ascon làm tiêu chuẩn cho mật mã nhẹ [4].

Ascon sử dụng mô hình thiết kế dựa trên cấu trúc sponge, cho phép xử lý dữ liệu theo cơ chế hấp thụ và trích xuất linh hoạt. Nhờ đó, Ascon có thể hỗ trợ nhiều chức năng mật mã khác nhau, nổi bật nhất là mã hóa kèm xác thực, còn được gọi là AEAD. Bên cạnh đó, Ascon được xây dựng trên mạng thay thế - hoán vị, đóng vai trò là nền tảng cho phép biến đổi nội bộ thuật toán [4], [8].

Thuật toán Ascon sở hữu các đặc tính quan trọng sau [4], [8], [13]:

- Single-pass: Xử lý dữ liệu được thực hiện đúng một lần duy nhất, giúp giảm độ trễ và chi phí tính toán.
- Online: Có thể xử lý dữ liệu dạng luồng mà không cần biết trước toàn bộ dữ liệu đầu vào.
- Inverse-free: Không yêu cầu các phép toán nghịch đảo.
- Security proof: Có thể chứng minh, đảm bảo mức độ an toàn trước các mô hình tấn công chuẩn.
- Lightweight: Tối ưu cho môi trường tài nguyên hạn chế về bộ nhớ, năng lượng và khả năng xử lý.
- Fast in software and hardware: Đạt được hiệu năng cao trên phần mềm và phần cứng.
- No table look-up: Không sử dụng bảng tra cứu, giảm nguy cơ rò rỉ dữ liệu thông qua các đòn tấn công kênh bên.

Trong phạm vi đề tài này, biến thể được sử dụng là Ascon-AEAD128, cung cấp mức bảo mật 128-bit và phù hợp với các ứng dụng yêu cầu đồng thời tính bảo mật và toàn vẹn dữ liệu.

### **2.3.2 Cấu trúc thuật toán Ascon**

Ascon được xây dựng trên mô hình sponge kết hợp hàm hoán vị nội bộ [4], [8]. Trong mô hình này, dữ liệu được “hấp thụ” vào trạng thái nội bộ và kết quả được “trích xuất” từ trạng thái sau khi thực hiện phép biến đổi.

Trạng thái nội bộ của Ascon có kích thước 320-bit, được biểu diễn dưới dạng [4]:

$$S = S_0 \parallel S_1 \parallel S_2 \parallel S_3 \parallel S_4$$

Trong đó, mỗi  $S_i$  là một từ 64 bit [4], [8]. Trạng thái này được cập nhật thông qua các phép biến đổi hoán vị lặp lại nhiều vòng.

Hàm hoán vị của Ascon (Ascon-p) được thiết kế theo dạng mạng thay thế-hoán vị (SPN) [4], [8]. Trong mỗi vòng, trạng thái của Ascon được biến đổi qua ba lớp hoạt động chính:

- Lớp cộng hằng số (Constant Addition Layer): Thêm 7 bit hằng số và trạng thái trong mỗi vòng nhằm phá vỡ tính đối xứng.
- Lớp thay thế phi tuyến (Substitution Layer): Sử dụng S-box 5-bit để tạo tính phi tuyến, giúp chống lại các tấn công tuyến tính và vi sai.
- Lớp khuếch tán tuyến tính (Linear Diffusion Layer): Thực hiện các phép xoay bit và XOR nhằm phân tán ảnh hưởng của từng bit trên toàn bộ trạng thái.

Cấu trúc thuật toán này giúp đảm bảo hai tính chất quan trọng trong mật mã hóa: nhiễu loạn và khuếch tán. Từ đó tăng cường khả năng chống lại các phương pháp phân tích mật mã hiện đại [4].

Trong quá trình hoạt động theo mô hình sponge, trạng thái sẽ được cập nhật xen kẽ giữa các bước hấp thụ dữ liệu (associated data và plaintext) và các lần áp dụng hàm hoán vị. Kết quả cuối cùng của quá trình này là bản mã và thẻ xác thực (authentication tag), đảm bảo cả tính bảo mật và toàn vẹn dữ liệu.

### 2.3.3 Nguyên lý hoạt động

Hình dưới đây minh họa quy trình mã hóa và giải mã của thuật toán Ascon-AEAD128, bao gồm 4 giai đoạn [4]:

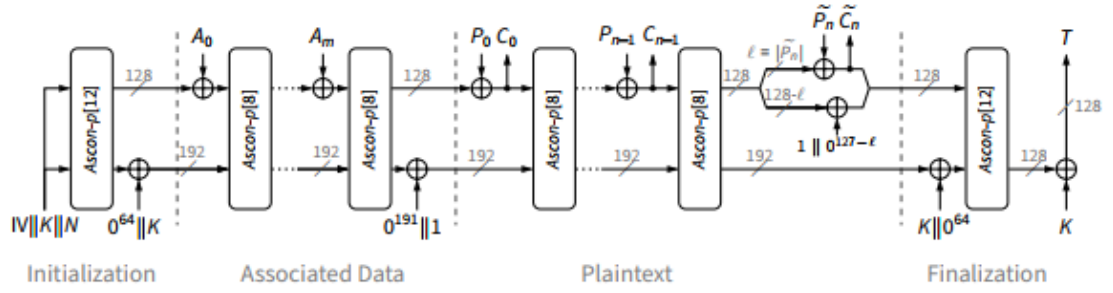


Figure 5. Ascon-AEAD128 encryption

Hình 2.5 Quy trình mã hóa thuật toán Ascon [4]

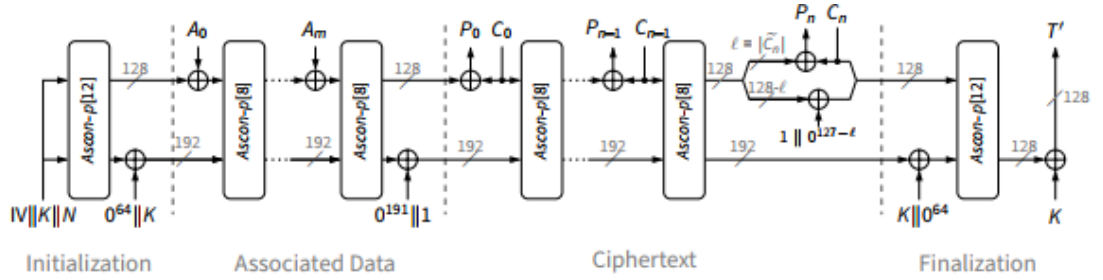


Figure 6. Ascon-AEAD128 decryption

Hình 2.6 Quy trình giải mã thuật toán Ascon [4]

Ở giai đoạn khởi tạo, trạng thái ban đầu được tạo ra từ khóa bí mật, nonce và một giá trị khởi tạo cố định IV. Sau đó được đưa vào hàm hoán vị Ascon-p[12] để tạo trạng thái nội bộ ban đầu. Việc thực hiện bước này nhằm đảm bảo rằng một phiên bản mã hóa là duy nhất khi nonce không bị lặp lại.

Sau khi trạng thái ban đầu được tạo ra, dữ liệu liên kết A được đưa vào trạng thái ban đầu thông qua phép chéo XOR và hoán vị. Mục đích của bước này nhằm đảm bảo tính toàn vẹn và xác thực của dữ liệu.

Ở giai đoạn mã hóa, dữ liệu plaintext được chia thành các khối và xử lý tuần tự, với mỗi khối ở trạng thái hiện tại XOR với plaintext để tạo ra ciphertext. Sau đó trạng thái được tiếp tục được cập nhật thông qua hàm hoán vị. Cơ chế này nhằm đảm bảo mỗi khối ciphertext đều phụ thuộc vào toàn bộ dữ liệu trước đó, tăng cường tính bảo mật. Sau khi xử lý toàn bộ dữ liệu, khóa bí mật được đưa lại vào trạng thái và thực hiện thêm các vòng hoán vị, tạo ra thẻ xác thực T, được gửi kèm theo ciphertext. Ở giai đoạn giải mã, quy trình diễn ra theo bước tương tự nhưng ngược chiều lại. Sau khi XOR và cập nhật trạng thái thông qua hoán vị,

sẽ tạo ra xác thực  $T'$  để so sánh với  $T$ . Nếu hai giá trị này không trùng khớp, dữ liệu sẽ bị từ chối nhằm đảm bảo tính toàn vẹn và chống giả mạo.

#### **2.3.4 Đánh giá**

Ascon được đánh giá là một trong những thuật toán mã hóa phù hợp nhất cho hệ thống nhúng hiện đại nhờ các ưu điểm sau:

- Hiệu năng cao: Ascon có thể đạt throughput khoảng 2.42 MB/s trên nền tảng ESP32 theo một số nghiên cứu thực nghiệm [5].
- Tiêu thụ tài nguyên thấp: Cấu trúc hoán vị giúp giảm số lượng phép tính phức tạp và yêu cầu bộ nhớ [5].
- Tích hợp AEAD: Không cần triển khai thêm cơ chế xác thực trên, giúp giảm độ phức tạp của hệ thống.
- Phù hợp với IoT: Hoạt động tốt trong môi trường hạn chế năng lượng và khả năng xử lý.

Tuy nhiên, Ascon vẫn tồn tại một số hạn chế như chưa được phổ biến rộng rãi như AES trong các hệ thống truyền thông, yêu cầu quản lý nonce nghiêm ngặt để tránh lỗ hổng bảo mật.

Trong bối cảnh đề tài, Ascon đóng vai trò là đại diện cho các thuật toán mã hạng nhẹ hiện đại, cho phép so sánh với AES và ChaCha20 về hiệu năng, mức sử dụng tài nguyên và khả năng đảm bảo an toàn dữ liệu. Cơ chế xác thực tích hợp giúp Ascon phù hợp với các ứng dụng yêu cầu bảo mật và toàn vẹn dữ liệu.

### **2.4 GIAO TIẾP UART**

UART là giao tiếp nối tiếp bất đồng bộ được sử dụng phổ biến trong nhiều hệ thống nhúng và Iot nhờ tính đơn giản, ổn định và khai triển dễ dàng [11], [16].

UART sử dụng hai đường dây truyền dữ liệu chính gồm TX (Transmit) và RX (Receive) để thực hiện việc truyền và nhận từ hai thiết bị [11], [16]. UART là kiểu giao tiếp truyền bất đồng bộ, nghĩa là không sử dụng xung đồng hồ để xác định điểm đầu và cuối của dữ liệu. Thay vào đó ở mỗi gói dữ liệu sẽ được thêm

một bit bắt đầu và một bit kết thúc để xác nhận việc bắt đầu và kết thúc của một gói dữ liệu khi truyền [16].

Dữ liệu trong UART được truyền theo từng khung dữ liệu, với mỗi khung dữ liệu có cấu trúc như sau: 1 bit bắt đầu, 8 bit dữ liệu, 1 bit kết thúc [16]. Khi đường truyền không hoạt động, mức tín hiệu luôn nằm ở mức cao. Khi bắt đầu truyền, bit bắt đầu có giá trị logic thấp, sau đó đến các bit dữ liệu, cuối cùng bit kết thúc được gửi để báo hiệu kết thúc khung dữ liệu [16].

UART có nhiều ưu điểm như cấu hình phần cứng đơn giản, dễ triển khai, hỗ trợ cho hầu hết các dòng vi điều khiển hiện nay [11], [16]. Tuy nhiên, UART cũng tồn tại một số hạn chế như tốc độ truyền thấp hơn SPI, chỉ hỗ trợ giao tiếp điểm điểm, khoảng cách truyền bị giới hạn nếu không sử dụng các mạch chuyển đổi tín hiệu hỗ trợ.

Trong đề tài, giao tiếp UART được sử dụng để truyền dữ liệu mã hóa giữa nút gửi và nút nhận. Nhờ độ ổn định của giao tiếp UART, việc đánh giá hiệu năng và độ tin cậy của các thuật toán AES, ChaCha20 và Ascon được thực hiện chính xác và ổn định hơn.

## CHƯƠNG 3

# THIẾT KẾ HỆ THỐNG TRIỂN KHAI, SO SÁNH VÀ ĐÁNH CÁC THUẬT TOÁN ĐỐI XỨNG

### 3.1 YÊU CẦU CỦA HỆ THỐNG

Hệ thống được thiết kế nhằm đánh giá hiệu năng và khả năng bảo mật của ba thuật toán mã hóa gồm AES, ChaCha20 và Ascon trong các kiểu truyền dữ liệu khác nhau. Các thuật toán được triển khai trên mô hình gồm một nút gửi và một nút nhận.

Cụ thể, hệ thống thực hiện ba kịch bản thử nghiệm như sau:

Kịch bản 1 – Truyền dữ liệu định kỳ: Mỗi thuật toán được triển khai độc lập trên một hệ thống, trong đó nút gửi thực hiện việc truyền dữ liệu theo chu kỳ xác định. Mục tiêu của kịch bản này là đánh giá hiệu năng của từng thuật toán trong điều kiện tải thấp, thông qua các chỉ số như thời gian mã hóa, giải mã và độ trễ truyền.

Kịch bản 2 – Truyền dữ liệu liên tục: Tương tự như kịch bản một, các thuật toán được triển khai độc lập trên một hệ thống, tuy nhiên dữ liệu được truyền liên tục với tần suất cao. Kịch bản này nhằm đánh giá hiệu năng thuật toán trong điều kiện tải cao, từ đó xác định khả năng đáp ứng thông lượng và độ ổn định của hệ thống.

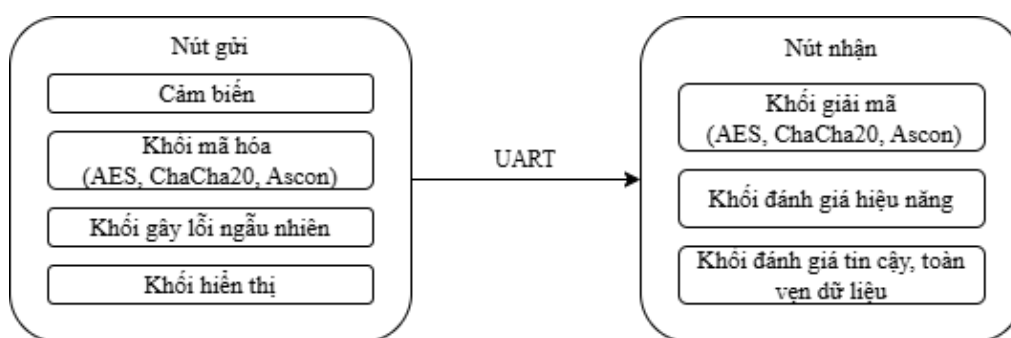
Kịch bản 3 – Đánh giá độ tin cậy và toàn vẹn dữ liệu của các thuật toán mã hóa trong điều kiện có lỗi: Cả ba thuật toán được tích hợp trong cùng một chương trình trên nút gửi và nhận. Dữ liệu được xử lý đồng thời qua các thuật toán nhằm kiểm tra khả năng phát hiện lỗi và đảm bảo tính toàn vẹn dữ liệu. Mục tiêu của kịch bản này là đánh giá mức độ tin cậy của từng thuật toán trong việc phát hiện sai lệch dữ liệu và chống lại các tác động gây lỗi trong quá trình truyền.



Thông qua ba kịch bản trên, hệ thống cho phép so sánh toàn diện các thuật toán về cả hiệu năng và mức bảo mật, từ đó đưa ra nhận xét và lựa chọn thuật toán phù hợp cho từng ứng dụng cụ thể.

### 3.2 MÔ HÌNH HỆ THỐNG

Với mục tiêu tạo ra một hệ thống triển khai, so sánh và đánh giá các thuật toán mã hóa đối xứng trên hệ thống nhúng tài nguyên hạn chế, hệ thống được xây dựng dưới dạng mô hình gồm hai nút hoạt động chính: nút gửi và nút nhận. Mô hình này cho phép triển khai và kiểm thử các thuật toán AES, ChaCha20 và Ascon trong cùng một điều kiện hoạt động, nhằm đảm bảo tính khách quan khi so sánh. Dưới đây là mô hình tổng thể của hệ thống.



Hình 3.1 Mô hình hệ thống

Theo như mô hình đã đề ra, hệ thống được tổ chức thành hai khối chính như sau:

- Nút gửi: Đảm nhận vai trò thu thập dữ liệu từ cảm biến, sau khi thu dữ liệu sẽ tiến hành mã hóa bằng một trong ba thuật toán AES, ChaCha20 và Ascon. Trong kịch bản truyền trong dữ liệu có nhiễu, nút gửi sẽ thực hiện gây lỗi ngẫu nhiên cho dữ liệu sau mã hóa để mô phỏng gói dữ liệu bị lỗi ngẫu nhiên trên đường truyền. Nút gửi sẽ có một màn hình dùng để hiển thị sơ lược những gì nút gửi làm. Sau khi thực hiện các việc trên, nút gửi sẽ gửi dữ liệu đến nút nhận thông qua giao tiếp UART.
- Nút nhận: Đảm nhận vai trò tiếp nhận và xử lý dữ liệu nhận được từ nút gửi. Sau khi nhận được dữ liệu, nút gửi sẽ thực hiện giải mã theo

thuật toán tương ứng, sau đó tính toán các thông số hiệu năng và thông số về độ tin cậy và toàn vẹn dữ liệu.

### 3.3 THIẾT KẾ PHẦN CỨNG

#### 3.3.1 Chức năng của phần cứng

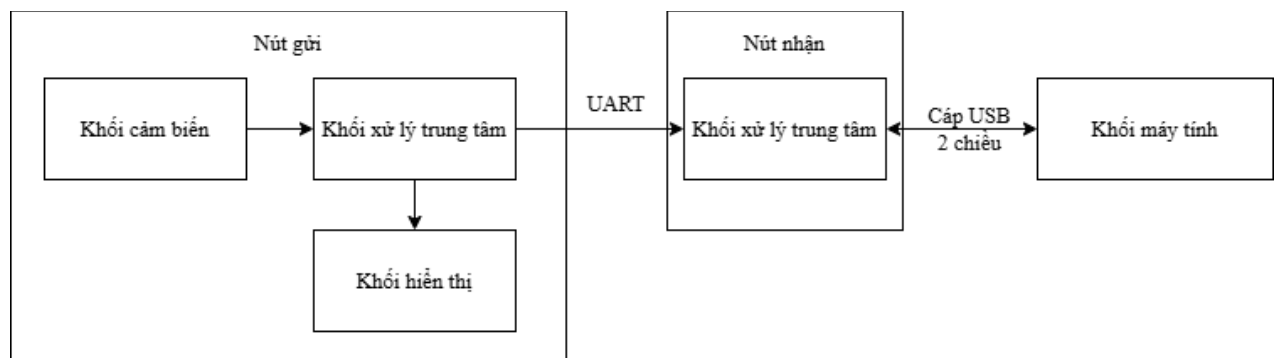
Dựa trên các yêu cầu của hệ thống và mô hình hệ thống đã trình bày ở các mục trên, phần cứng của hệ thống cần được thiết kế để đáp ứng các chức năng sau:

Chức năng nút gửi: Phần cứng phải có khả năng thu thập dữ liệu từ môi trường bên ngoài, có khả năng mã hóa dữ liệu bằng các thuật toán mã hóa AES, ChaCha20 và Ascon. Ngoài ra, phần cứng phải hỗ trợ giao tiếp với các khối ngoại vi, hỗ trợ hiển thị trạng thái hoạt động của hệ thống và hỗ trợ các giao thức truyền thông để trao đổi dữ liệu với nút nhận. Và phần cứng phải đáp ứng được việc triển khai nhiều kịch bản thử nghiệm khác nhau để phục vụ cho việc so sánh và đánh giá.

Chức năng nút nhận: Phần cứng phải có khả năng nhận dữ liệu từ nút gửi, thực hiện giải mã theo thuật toán tương ứng. Bên cạnh đó, phần cứng phải có đủ năng lực tính toán để thực hiện đo đạc, thống kê và đánh giá các tiêu chí hiệu năng thuật toán, hiệu suất hệ thống, độ tin cậy và toàn vẹn dữ liệu. Ngoài ra, hệ thống phải có khả năng xuất dữ liệu để quan sát, thu thập và phân tích số liệu.

#### 3.3.2 Sơ đồ khối phần cứng

Dựa trên mô hình hệ thống được xây dựng ở mục 3.2, dưới đây là ảnh sơ đồ khối thực tế của hệ thống:



Hình 3.2 Sơ đồ khối phần cứng của hệ thống thực nghiệm

### 3.3.3 Thiết kế từng khối

#### 3.3.3.1 Khối cảm biến

Khối cảm biến đảm nhận chức năng ghi nhận tín hiệu từ môi trường xung quanh và chuyển đổi nó từ tín hiệu tương tự sang tín hiệu số, sau đó đưa tín hiệu ghi nhận được cho bộ xử lý trung tâm để xử lý.

Dưới đây là bảng so sánh một số thiết bị cảm biến đo được cả nhiệt độ và độ ẩm [7], [9], [10]:

**Bảng 3.1** So sánh các linh kiện cảm biến

| Tiêu chí               | DHT22                       | SHT30                     | BME280                   |
|------------------------|-----------------------------|---------------------------|--------------------------|
| <i>Đại lượng đo</i>    | Nhiệt độ, độ ẩm             | Nhiệt độ, độ ẩm           | Nhiệt độ, độ ẩm, áp suất |
| <i>Sai số nhiệt độ</i> | $\pm 0,5^{\circ}\text{C}$   | $\pm 0,3^{\circ}\text{C}$ | $\pm 1^{\circ}\text{C}$  |
| <i>Sai số độ ẩm</i>    | $\pm 2 \div 5\% \text{ RH}$ | $\pm 2\% \text{ RH}$      | $\pm 3\% \text{ RH}$     |
| <i>Giao tiếp</i>       | Digital                     | I2C                       | I2C/SPI                  |
| <i>Giá thành [21]</i>  | 36.480đ                     | 112.860đ                  | 364.800đ                 |

Qua bảng 3.1, có thể thấy sau khi so sánh các cảm biến, đề tài quyết định chọn DHT22 là khối cảm biến chính cho hệ thống. Vì DHT22 có độ chính xác đáp ứng được yêu cầu của hệ thống, giao tiếp đơn giản, giá thành hợp lý.

#### Giới thiệu DHT22

DHT22 là một cảm biến nhiệt độ và độ ẩm kỹ thuật số có độ chính xác cao, được phát triển bởi Aosong Electronics và được sử dụng rộng rãi trong các hệ thống IoT và giám sát môi trường. Cảm biến cung cấp dữ liệu đã được hiệu chỉnh dưới dạng tín hiệu số, giúp giảm thiểu sai số và đơn giản hóa quá trình xử lý dữ liệu [7].

DHT22 thực hiện đo nhiệt độ và độ ẩm thông qua cảm biến nội và chuyển đổi thành dữ liệu số. Khi nhận được tín hiệu khởi tạo từ vi điều khiển, cảm biến sẽ gửi về một khung dữ liệu gồm 40 bit, bao gồm dữ liệu nhiệt độ, độ ẩm và byte

kiểm tra lỗi [7]. Việc truyền dữ liệu được thực hiện theo giao thức một dây, yêu cầu tín hiệu có độ ổn định cao.

### Sơ đồ chân DHT22

Dưới đây là ảnh thực tế của cảm biến DHT22:

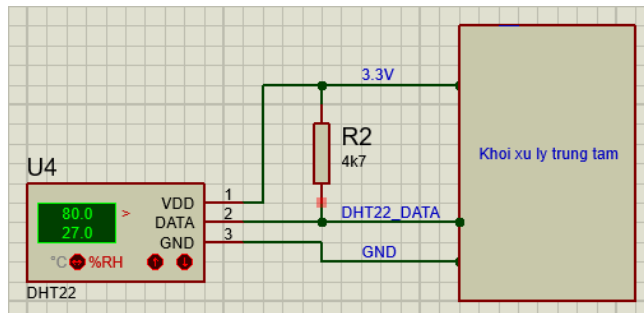


Hình 3.3 Cảm biến DHT22 [21]

Cảm biến DHT22 có tổng cộng 4 chân vật lý, bao gồm chân nguồn Vcc, chân dữ liệu DATA, chân không sử dụng NC và chân đất GND.

### Sơ đồ nguyên lý kết nối

Dưới đây là sơ đồ nguyên lý kết nối giữa cảm biến DHT22 và khối xử lý trung tâm:



Hình 3.4 Sơ đồ nguyên lý của cảm biến DHT22

Để đảm bảo tín hiệu truyền ổn định, một điện trở kéo lên được sử dụng giữa chân DATA và nguồn 3.3V với giá trị  $4.7k\Omega$  theo khuyến nghị của nhà sản xuất [7].

#### 3.3.3.2 Khối hiển thị

Khối hiển thị có chức năng hỗ trợ giám sát trạng thái hoạt động của nút gửi theo thời gian thực. Việc tích hợp thêm màn hình giúp thuận tiện hơn trong quá trình hoạt động mà không cần thông qua Serial Monitor.

Dưới đây là bảng so sánh một số màn hình phù hợp với đề tài [15], [24], [25]:

**Bảng 3.2** So sánh các màn hình

| Tiêu chí                         | LCD 16x2                | OLED<br>SSD1306  | TFT LCD<br>ILI9341        |
|----------------------------------|-------------------------|------------------|---------------------------|
| <i>Hiển thị kí tự</i>            | Có                      | Có               | Có                        |
| <i>Hiển thị đồ họa</i>           | Hạn chế                 | Có               | Có                        |
| <i>Giao tiếp</i>                 | I2C/Paralled            | I2C              | SPI                       |
| <i>Kích thước</i>                | 80.0 x 36.0 x<br>13.5mm | 30 x 28 x<br>4mm | 89.30 x 55.04<br>x 15.9mm |
| <i>Độ phức tạp khi lập trình</i> | Thấp                    | Thấp             | Cao                       |
| <i>Giá thành [21]</i>            | 34.200đ                 | 57.000đ          | 265.620đ                  |

Qua bảng 3.2, sau khi lựa chọn kĩ càng, màn hình OLED SSD1306 được lựa chọn cho hệ thống. Màn hình có kích thước nhỏ, hỗ trợ I2C, độ khó lập trình thấp, giá thành hợp lí.

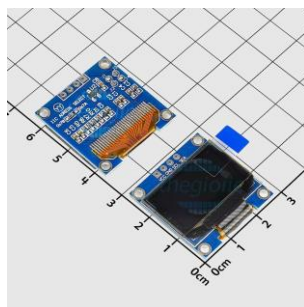
#### **Giới thiệu màn hình OLED SSD1306**

Màn hình OLED SSD1306 là một loại màn hình hiển thị được sử dụng phổ biến trong các hệ thống nhúng và Internet of Things (IoT) nhờ kích thước nhỏ gọn, độ tương phản cao, khả năng hiển thị rõ nét và mức tiêu thụ năng lượng thấp. Màn hình có độ phân giải 128×64 điểm ảnh và sử dụng bộ điều khiển SSD1306 để quản lý việc hiển thị dữ liệu [16].

OLED SSD1306 sử dụng giao tiếp I2C để trao đổi dữ liệu với khối xử lý trung tâm thông qua đường tín hiệu SDA và SCL [16]. Dữ liệu hiển thị được lưu trữ trong bộ nhớ đồ họa GDDRAM bên trong SSD1306 [16]. Sau khi nhận được dữ liệu, bộ điều khiển SSD1306 sẽ cập nhật trạng thái các điểm ảnh tương ứng trên màn hình OLED.

#### **Sơ đồ chân OLED SSD1306**

Dưới đây là ảnh màn hình OLED SSD1306 thực tế:

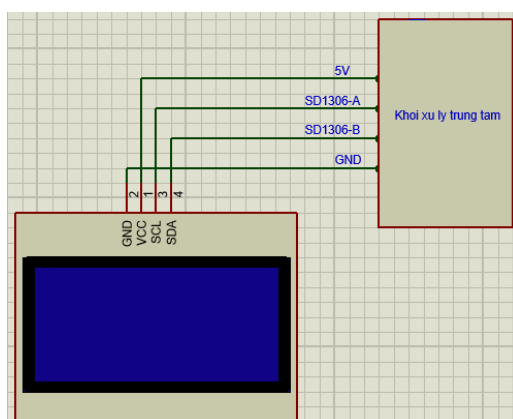


Hình 3.5 Màn hình OLED SSD1306 [21]

Màn hình OLED SSD1306 có tổng cộng 4 chân kết nối gồm VCC, GND, SDA và SCL. Trong đó, SDA là đường truyền dữ liệu và SCL là đường tín hiệu xung nhịp của giao tiếp I2C.

### Sơ đồ nguyên lý kết nối

Dưới đây là sơ đồ nguyên lý kết nối giữa màn hình với khối xử lý trung tâm:



Hình 3.6 Sơ đồ nguyên lý của màn hình OLED SSD1306

Màn hình OLED SSD1306 được kết nối với khối xử lý trung tâm thông qua giao tiếp I2C. Chân SDA và SCL được kết nối với chân I2C của khối xử lý trung tâm. VCC và GND được sử dụng để cấp nguồn và nối mass cho màn hình.

### 3.3.3.3 Khối xử lý trung tâm

Khối xử lý trung tâm đóng vai trò điều khiển và quản lý mọi hoạt động của hệ thống. Khối này phải có khả năng sử dụng các thư viện mã hóa AES-128-CBC, ChaCha20 và Ascon-128. Có hỗ trợ giao tiếp một dây cho cảm biến DHT22, giao tiếp I2C cho màn hình OLED SSD1306, giao tiếp UART giữa nút gửi và nút nhận. Có khả năng cung cấp các mức điện áp 3.3V và 5V ổn định cho các khối chức năng trong hệ thống.

Để lựa chọn vi điều khiển phù hợp cho hệ thống, một số nền tảng vi điều khiển được lựa chọn để so sánh với các tiêu chí của đề tài. Dưới đây là kết quả so sánh [6], [21], [22]:

**Bảng 3.3** So sánh các vi điều khiển

| Tiêu chí                               | Arduino<br>UNO | ESP8266<br>NodeMCU | ESP32<br>NodeMCU |
|----------------------------------------|----------------|--------------------|------------------|
| <i>Hỗ trợ các thư viện thuật toán</i>  | Hạn chế        | Có                 | Có               |
| <i>Hỗ trợ giao tiếp một dây</i>        | Có             | Có                 | Có               |
| <i>Hỗ trợ giao tiếp I2C</i>            | Có             | Có                 | Có               |
| <i>Hỗ trợ giao tiếp UART</i>           | Có             | Có                 | Có               |
| <i>Mức điện áp 3.3V và 5V</i>          | Có             | Hạn chế            | Có               |
| <i>Giám sát serial monitor dễ dàng</i> | Có             | Có                 | Có               |
| <i>Dễ nạp chương trình liên tục</i>    | Có             | Có                 | Có               |
| <i>Phù hợp với đề tài</i>              | Trung bình     | Khá                | Cao              |

Qua bảng so sánh 3.3, ESP32 NodeMCU là lựa chọn phù hợp với đề tài nhất so với hai vi xử lý hiện tại. Vì có hỗ trợ các giao tiếp cần thiết, có cung cấp các chân mức điện áp 3.3V và 5V, dễ dàng quan sát và nạp bằng Arduino IDE.

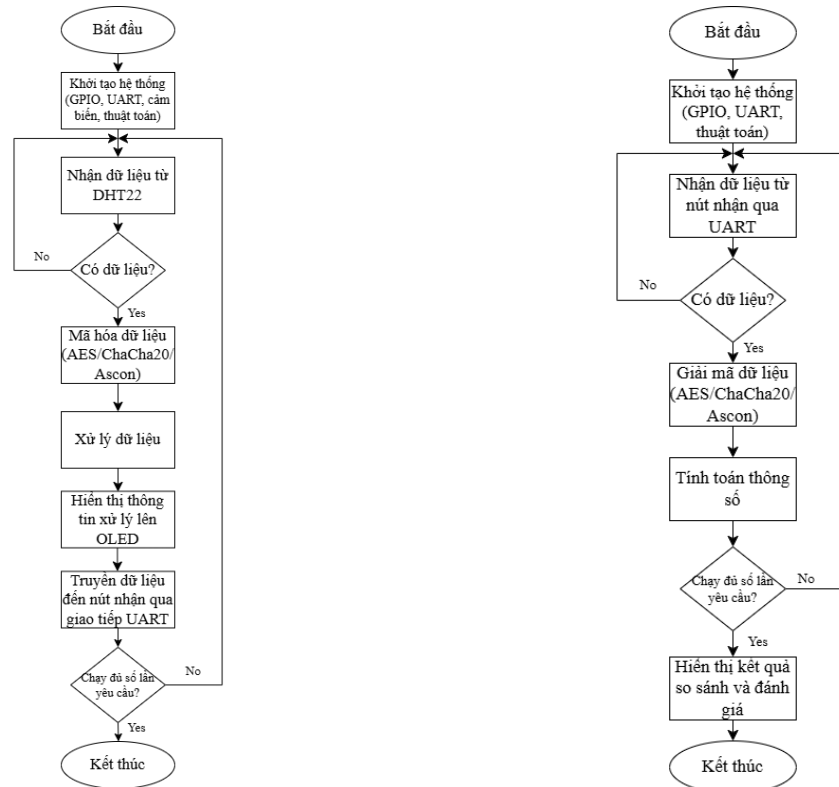
#### **Giới thiệu ESP32 NodeMCU**

ESP32 NodeMCU là một bo mạch phát triển dựa trên vi điều khiển ESP32, được thiết kế nhằm hỗ trợ nhanh chóng cho các ứng dụng Internet of Things (IoT) và hệ thống nhúng. Bo mạch này tích hợp sẵn module ESP32 cùng với các thành phần hỗ trợ như mạch nguồn, giao tiếp USB và chân GPIO được đưa ra ngoài, giúp việc lập trình và kết nối ngoại vi trở nên thuận tiện hơn [6].

ESP32 NodeMCU hỗ trợ các kết nối Wi-Fi, Bluetooth, nhiều giao tiếp ngoại vi như UART, SPI, I2C và có năng lực xử lý phù hợp cho các ứng dụng IoT và hệ thống nhúng. Trong đề tài này, ESP32 NodeMCU được dùng làm khối xử lý trung tâm ở nút gửi và nút nhận.

## Nguyên lí hoạt động

Để làm rõ hơn trình tự xử lý dữ liệu trong hệ thống, lưu đồ hoạt động của khối xử lý ESP32 được xây dựng nhằm mô tả chi tiết các bước thực hiện ở phía gửi và phía nhận như sau:



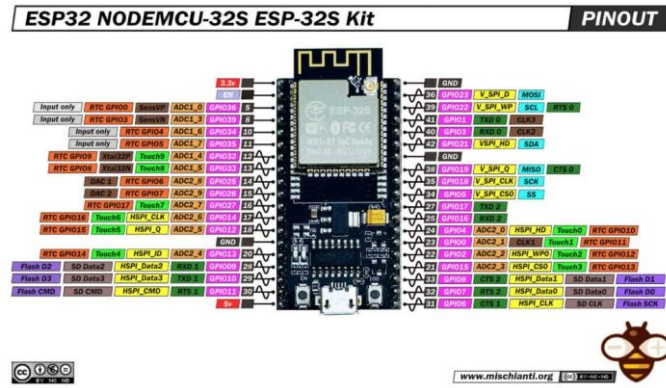
Hình 3.7 Lưu đồ hoạt động của ESP32 phía gửi và phía nhận

Lưu đồ hình 3.4 mô tả quy trình hoạt động của ESP32 phía gửi và phía nhận trong hệ thống. ESP32 phía gửi sẽ thực hiện việc thu thập dữ liệu từ cảm biến DHT22, sau đó xử lý và mã hóa dữ liệu bằng các thuật toán AES, ChaCha20, Ascon trước khi truyền đến ESP32 phía nhận. Sau khi nhận được dữ liệu từ ESP32 phía gửi, ESP32 phía nhận sẽ thực hiện giải mã dữ liệu theo thuật toán tương ứng, sau đó tổng kết các thông số cần thiết để phục vụ cho việc đánh giá hiệu năng và độ toàn vẹn của thuật toán.

## Sơ đồ chân ESP32 NodeMCU

Nút nhận và nút gửi đều xài cùng một loại ESP32 NodeMCU có sơ đồ chân như sau:



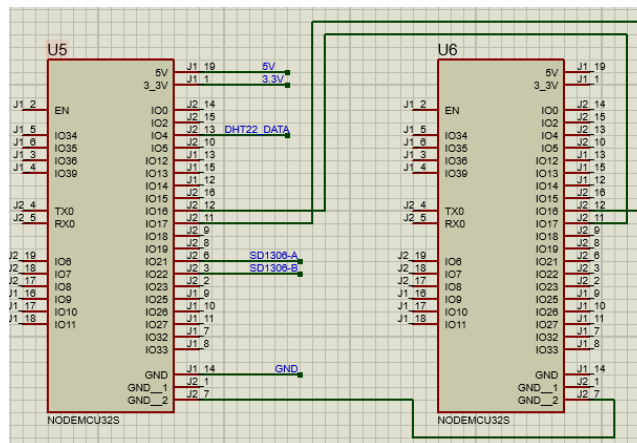


Hình 3.8 Sơ đồ chân của ESP32 NodeMCU [26]

ESP32 NodeMCU cung cấp nhiều chân GPIO cấu hình linh hoạt cho các giao tiếp UART, SPI, I2C và một dây [6]. Trong đề tài này, GPIO4 được sử dụng để giao tiếp với cảm biến DHT22, GPIO21 và GPIO22 được sử dụng cho giao tiếp I2C của màn hình OLED SSD1306, GPIO16 và GPIO17 được sử dụng cho giao tiếp UART giữa nút gửi và nút nhận.

### Sơ đồ nguyên lý kết nối

Dưới đây là sơ đồ nguyên lý kết nối giữa hai khối xử lý trung tâm của nút gửi và nút nhận:

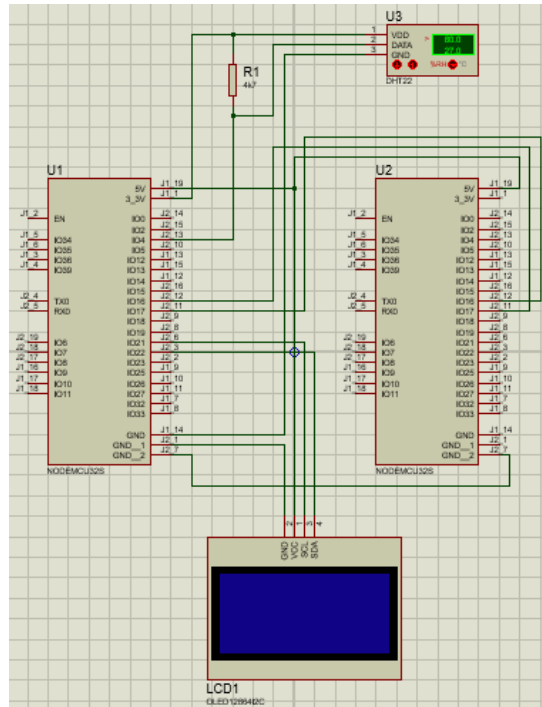


Hình 3.9 Sơ đồ nguyên lý của khối xử lý trung tâm của nút gửi và nút nhận

Hai khối xử lý trung tâm của nút gửi và nút nhận sẽ được kết nối với nhau thông qua hai chân GPIO16 và GPIO17 của giao tiếp UART.

### 3.3.3.4 Sơ đồ nguyên lý toàn hệ thống

Dưới đây là sơ đồ nguyên lý của toàn bộ hệ thống:



Hình 3.10 Mạch sơ đồ nguyên lý

### 3.4 THIẾT KẾ KỊCH BẢN SO SÁNH VÀ ĐÁNH GIÁ THUẬT TOÁN

#### 3.4.1 Thiết kế kịch bản thử nghiệm số 1: Nút cảm biến truyền dữ liệu định kỳ

##### Mô tả kịch bản

Kịch bản thử nghiệm số 1 mô tả hành vi truyền dữ liệu định kỳ từ nút gửi đến nút nhận. Dữ liệu từ cảm biến DHT22 được thu thập, mã hóa và truyền qua giao tiếp UART. Tại nút nhận, dữ liệu được giải mã và xử lý để tính toán các thông số hiệu năng. Kịch bản này nhằm đánh giá hiệu năng của các thuật toán AES, ChaCha20 và Ascon trong điều kiện tải thấp và ổn định.

##### Tiêu chí đánh giá

Kịch bản đánh giá các thuật toán thông qua các tiêu chí sau:

- Thời gian mã hóa và giải mã (trung bình, nhỏ nhất, lớn nhất).
- Thời gian truyền UART (trung bình, nhỏ nhất, lớn nhất).
- Thời gian độ trễ toàn hệ thống (trung bình, nhỏ nhất, lớn nhất).
- Throughput thuật toán.
- Throughput hệ thống.

- Dung lượng Flash sử dụng.
- Dung lượng RAM sử dụng.

### **Cấu hình thử nghiệm**

- Mỗi thuật toán được triển khai độc lập trên một hệ thống.
- Kích thước dữ liệu: 32 bytes.
- Giao tiếp truyền thông: UART.
- Số lần truyền: 200 lần.
- Nền tảng phần cứng: ESP32.
- Tốc độ truyền: 115200 bps.

### **Phương pháp thử nghiệm và tính toán**

Thời gian mã hóa và thời gian giải mã được đo trực tiếp trên vi điều khiển ESP32 bằng hàm `micros()`, với độ phân giải micro giây (us). Giá trị thời gian được xác định bằng hiệu giữa thời điểm trước và sau khi thực hiện hàm mã hóa hoặc giải mã.

Thời gian truyền UART được xác định tại nút nhận dựa trên khoảng thời gian từ khi bắt đầu nhận được byte đầu tiên đến khi nhận đầy đủ gói dữ liệu, dùng đơn vị thời gian micro giây (us). Phương pháp này phù hợp với cơ chế UART không đồng bộ và sử dụng buffer trên ESP32, giúp giảm sai số khi đo thời gian truyền.

Do hai nút gửi và nút nhận không sử dụng chung một nguồn clock, thời gian độ trễ toàn hệ thống không thể xác định trực tiếp thông qua timestamp tuyệt đối. Vì vậy, thời gian độ trễ toàn hệ thống được xác định bằng tổng thời gian mã hóa, thời gian truyền và thời gian giải mã, đơn vị micro giây (us).

Thử nghiệm được thực hiện 200 lần liên tục để tăng độ tin cậy của thống kê và giảm ảnh hưởng của nhiễu hệ thống.

Về thông lượng được chia làm hai loại: Thông lượng thuật toán và thông lượng hệ thống. Thông lượng thuật toán phản ánh tốc độ xử lý của riêng thuật toán khi chạy trên phần cứng. Thông lượng hệ thống phản ánh hiệu năng tổng thể của thuật toán khi tích hợp vào ứng dụng cụ thể.

Thông lượng thuật toán:

$$\text{Throughput\_crypto} = \frac{\text{DATA SIZE} * 10^6}{T_{\text{avgenc}} * 1024} \quad (3.2)$$

Thông lượng hệ thống:

$$\text{Throughput\_system} = \frac{\text{TOTAL DATA} * 10^6}{T_{\text{avg2e}} * 1024} \quad (3.3)$$

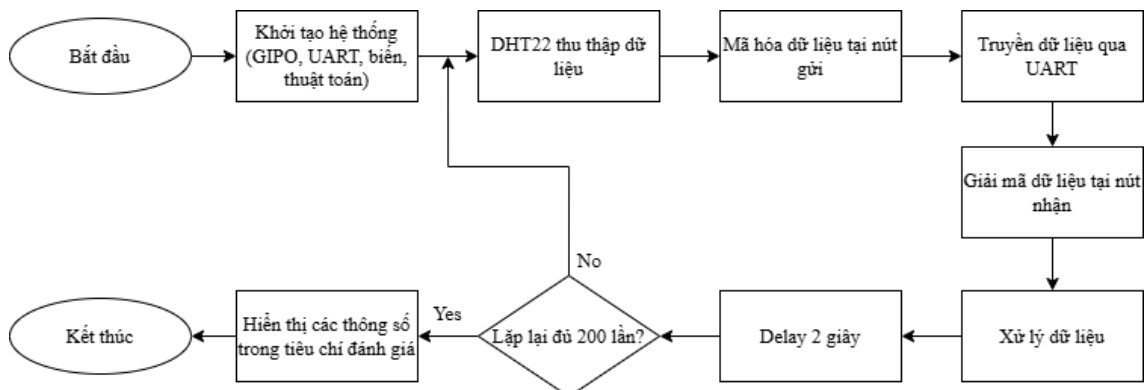
Với:

- Đơn vị throughput là KB/s.
- DATA SIZE: Kích thước gói dữ liệu, đơn vị là byte.
- TOTAL DATA: Tổng dữ liệu mỗi lần truyền, đơn vị là byte.
- $T_{\text{avgenc}}$ : Thời gian mã hóa trung bình sau 200 lần, đơn vị là us.
- $T_{\text{avg2e}}$ : Thời gian độ trễ toàn hệ thống trung bình sau 200 lần, đơn vị là us.

Dung lượng Flash được xác định thông qua kích thước chương trình sau khi biên dịch trên Arduino IDE, bao gồm mã nguồn và các thư viện liên quan. Dung lượng RAM được xác định dựa trên bộ nhớ tĩnh được cấp phát cho biến toàn cục và dữ liệu sau khi biên dịch, chưa bao gồm bộ nhớ cấp phát động trong quá trình thực thi.

### Lưu đồ giải thuật và quy trình thử nghiệm

Dưới đây là lưu đồ giải thuật của kịch bản 1: Nút cảm biến truyền dữ liệu định kỳ.



Hình 3.11 Lưu đồ giải thuật của kịch bản 1

Ban đầu, ESP32 khởi tạo các thành phần cần thiết gồm cảm biến DHT22, giao tiếp UART và thuật toán mã hóa tương ứng (AES, ChaCha20 hoặc Ascon). Kế tiếp, tại nút gửi, dữ liệu nhiệt độ và độ ẩm được thu thập từ DHT22, sau đó được mã hóa và ghi nhận thời gian mã hóa. Dữ liệu đã mã hóa sẽ được truyền đến nút nhận qua giao tiếp UART. Thời gian truyền được xác định tại phía nhận.

Sau khi nhận dữ liệu, nút nhận thực hiện giải mã bằng thuật toán tương ứng và ghi nhận thời gian giải mã. Các thông số thời gian xử lý được lưu lại để tính toán giá trị trung bình, lớn nhất, nhỏ nhất, thông lượng thuật toán và thông lượng hệ thống. Khoảng cách mỗi lần truyền là 2 giây, để xem xét khả năng xử lý dữ liệu tải thấp của các thuật toán.

Tổng thể, lưu đồ giải thuật trong Hình 3.11 thể hiện rõ tính tuần tự và lặp lại của quy trình thử nghiệm, đồng thời đảm bảo tất cả các bước từ thu thập dữ liệu đến đánh giá hiệu năng đều được thực hiện một cách có hệ thống. Điều này giúp đảm bảo tính chính xác, khách quan và khả năng tái lập của kết quả thử nghiệm trong quá trình so sánh các thuật toán mã hóa trên nền tảng ESP32.

### **3.4.2 Thiết kế kịch bản thử nghiệm số 2: Truyền dữ liệu liên tục**

#### **Mô tả kịch bản**

Kịch bản thử nghiệm số 2 mô tả hành vi truyền dữ liệu liên tục từ nút gửi đến nút nhận. Nút gửi sẽ tạo dữ liệu nhiệt độ và độ ẩm ngẫu nhiên, sau đó tiến hành mã hóa dữ liệu và gửi đến nút nhận thông qua giao tiếp UART. Nút nhận sau khi nhận được dữ liệu từ nút gửi thông qua giao tiếp UART sẽ tiến hành xử lý dữ liệu, bao gồm giải mã hóa, tính toán các thông số hiệu suất.

Khác với kịch bản 1 truyền định kỳ, kịch bản 2 loại bỏ các khoảng thời gian chờ, buộc các khối mã hóa và giao tiếp UART phải hoạt động liên tục. Điều này giúp đánh giá giới hạn thông lượng thực tế của hệ thống và khả năng duy trì ổn định của các thuật toán AES, ChaCha20 và Ascon trong điều kiện tải cao.

#### **Tiêu chí đánh giá**

Trong kịch bản này, các thuật toán được đánh giá dựa trên các tiêu chí chính sau:

- Số lượng gói dữ liệu được xử lý trong thời gian thử nghiệm.

- Thời gian mã hóa và giải mã (trung bình, nhỏ nhất, lớn nhất).
- Thời gian truyền (trung bình, nhỏ nhất, lớn nhất).
- Thời gian độ trễ hệ thống (trung bình, nhỏ nhất, lớn nhất).
- Thông lượng thuật toán, thông lượng hệ thống, thông lượng thực nghiệm.
- Chi phí xử lý trung bình trên mỗi byte dữ liệu.
- Dung lượng Flash và RAM.

### **Cấu hình thử nghiệm**

- Mỗi thuật toán được triển khai độc lập ở mỗi hệ thống.
- Kích thước dữ liệu: 32 bytes.
- Thời gian chạy thử nghiệm: 3 phút.
- Nền tảng phần cứng: ESP32.
- Giao tiếp truyền thông: UART.
- Tốc độ truyền: 115200 bps.

### **Phương pháp thử nghiệm và tính toán**

Trong quá trình thử nghiệm, các thông số thời gian xử lý được đo trực tiếp trên vi điều khiển ESP32 bằng hàm `micros()`, độ phân giải micro giây (us). Thời gian mã hóa và giải mã được xác định bằng cách ghi nhận ở thời điểm trước và sau khi thực hiện hàm mã hóa và giải mã tương ứng.

Thời gian truyền UART được xác định tại nút nhận dựa trên khoảng thời gian từ khi bắt đầu nhận dữ liệu đến khi nhận hoàn chỉnh một gói dữ liệu. Cách đo đặc sẽ giống với kịch bản 1 truyền định kỳ ở trên. Thời gian độ trễ toàn hệ thống cũng được xác định bằng phương pháp đã trình bày ở kịch bản 1, bằng tổng thời gian mã hóa, giải mã và thời gian truyền.

Trong điều kiện truyền liên tục, thời gian UART chiếm phần lớn thời gian trong tổng độ trễ hệ thống. Do đó, kênh truyền trở thành yếu tố giới hạn thông lượng tổng thể, làm giảm sự khác biệt về hiệu năng giữa các thuật toán mã hóa.

Sau 3 phút thử nghiệm, các giá trị thời gian sẽ được tổng hợp dưới dạng các giá trị trung bình, lớn nhất và nhỏ nhất nhằm phản ánh đầy đủ đặc tính hiệu năng

của thuật toán. Ngoài ra, hệ thống còn ghi nhận số lượng gói dữ liệu được xử lý thành công tại nút nhận. Thông số này phản ánh khả năng xử lý liên tục của hệ thống và được dùng để tính tổng dữ liệu truyền cũng như các chỉ số thông lượng.

Về thông lượng, trong kịch bản truyền liên tục, thông lượng được chia thành ba góc độ để đánh giá: thông lượng thuật toán, thông lượng hệ thống và thông lượng thực nghiệm. Thông lượng thuật toán được xác định dựa trên thời gian mã hóa trung bình, phản ánh năng lực xử lý của riêng thuật toán. Thông lượng hệ thống được tính toán dựa trên độ trễ toàn hệ thống, thể hiện hiệu năng tổng thể khi tích hợp vào các ứng dụng thực tế. Trong khi đó, thông lượng thực nghiệm được xác định dựa trên tổng dữ liệu truyền được trong toàn bộ quá trình thử nghiệm, cho phép đánh giá hiệu quả truyền tải thực tế của hệ thống trong điều kiện hoạt động liên tục. Khác với thông lượng thuật toán và thông lượng hệ thống, chỉ số này phản ánh trực tiếp khả năng xử lý thực tế khi xét đồng thời cả chi phí tính toán và giới hạn kênh truyền, do đó có ý nghĩa gần với điều kiện triển khai thực tế hơn.

Thông lượng thuật toán:

$$\text{Throughput\_crypto} = \frac{\text{DATA SIZE} * 10^6}{T_{\text{avgenc}} * 1024} \quad (3.4)$$

Thông lượng hệ thống:

$$\text{Throughput\_system} = \frac{\text{TOTAL DATA} * 10^6}{T_{\text{avge2e}} * 1024} \quad (3.5)$$

Thông lượng thực nghiệm:

$$\text{Throughput\_empirical} = \frac{\text{Total Bytes}}{\text{Duration} * 1024} \quad (3.6)$$

Với:

- Đơn vị throughput là KB/s.
- DATA SIZE: Kích thước gói dữ liệu, đơn vị là byte.
- TOTAL DATA: Tổng dữ liệu mỗi lần truyền, đơn vị là byte.
- Total Bytes: Tổng kích thước dữ liệu truyền được, đơn vị là byte.

- $T_{avgenc}$ : Thời gian mã hóa trung bình sau 3 phút, đơn vị là us.
- $T_{avge2e}$ : Thời gian độ trễ toàn hệ thống trung bình sau 3 phút, đơn vị là us
- Duration: Thời gian hoạt động của thí nghiệm, đơn vị là giây.

Ngoài ra, để đánh giá chi tiết hơn hiệu năng xử lý, chi phí trung bình trên mỗi byte dữ liệu cũng được tính toán dựa trên thời gian mã hóa và giải mã trung bình. Thông số này cho phép so sánh mức độ hiệu quả tính toán của thuật toán một cách độc lập với kích thước dữ liệu.

Chi phí mỗi byte:

$$Cost_{byte} = \frac{T_{process}}{DATA\ SIZE} \quad (3.7)$$

Với:

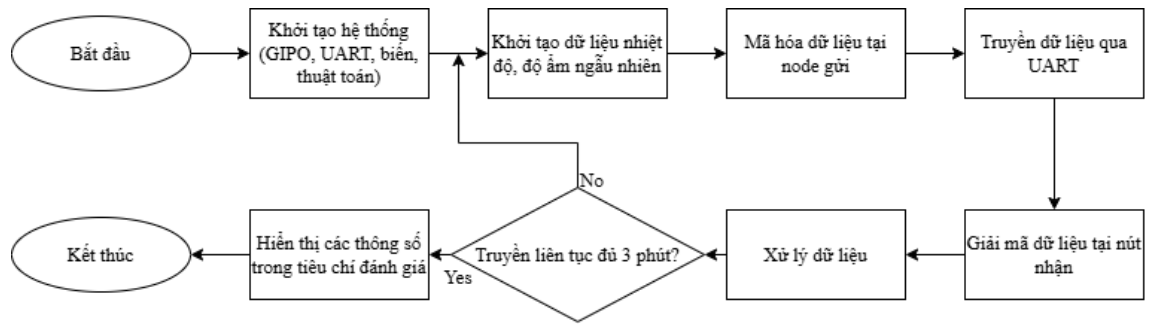
- $Cost_{byte}$ : Chi phí xử lý trung bình mỗi byte, đơn vị là us/byte.
- $T_{process}$ : Thời gian xử lý dữ liệu trung bình, đơn vị là us.
- DATA SIZE: Kích thước gói dữ liệu, đơn vị là byte.

Mặc dù thông lượng thuật toán và chi phí xử lý trung bình trên mỗi byte có mối quan hệ nghịch đảo và đều phản ánh hiệu năng xử lý, hai thông số này được sử dụng nhằm phục vụ các mục tiêu phân tích khác nhau. Thông lượng thuật toán thể hiện tốc độ xử lý tổng thể của thuật toán, trong khi chi phí xử lý trên mỗi byte phản ánh mức độ hiệu quả tính toán của thuật toán trên từng đơn vị dữ liệu. Việc sử dụng đồng thời hai chỉ số này cho phép đánh giá hiệu năng một cách toàn diện hơn, đặc biệt trong các trường hợp thay đổi kích thước dữ liệu hoặc so sánh khả năng mở rộng của hệ thống.

### **Lưu đồ giải thuật và quy trình thử nghiệm**

Dưới đây là lưu đồ giải thuật của kịch bản 2: Truyền dữ liệu liên tục.





Hình 3.12 Lưu đồ giải thuật của kịch bản 2

Ban đầu, hệ thống khởi tạo GPIO, giao tiếp UART, các thuật toán mã hóa và các biến cần thiết. Tại nút gửi, dữ liệu nhiệt độ và độ ẩm sau đó được tạo ngẫu nhiên để hạn chế hiện tượng dữ liệu lặp lại giữa các gói truyền. Sau đó dữ liệu được mã hóa và truyền liên tục qua UART đến nút nhận. Sau khi nhận dữ liệu, nút nhận thực hiện giải mã và ghi nhận các thông số như thời gian xử lý, số lượng gói dữ liệu và thông lượng.

Quy trình được lặp lại liên tục trong 3 phút. Sau mỗi chu kỳ, hệ thống kiểm tra thời gian hoạt động để quyết định tiếp tục thử nghiệm hoặc chuyển sang bước tổng hợp kết quả.

Trong suốt quá trình vận hành, các thông số gồm thời gian mã hóa, giải mã, truyền UART và số lượng gói dữ liệu được ghi nhận liên tục. Sau khi kết thúc thử nghiệm, các dữ liệu này được tổng hợp để tính toán các giá trị thống kê và chỉ số hiệu năng của hệ thống.

Tổng thể, lưu đồ giải thuật trong Hình 3.12 thể hiện rõ tính tuần tự và lặp lại của quy trình thử nghiệm. Thông qua kịch bản thử nghiệm này, có thể xác định được ảnh hưởng của cả thuật toán mã hóa và kênh truyền đến hiệu năng tổng thể, từ đó làm cơ sở cho việc lựa chọn giải pháp phù hợp trong các ứng dụng thực tế, đặc biệt trên nền tảng ESP32.

### 3.4.3 Thiết kế kịch bản thử nghiệm số 3: Đánh giá độ tin cậy và toàn vẹn dữ liệu của các thuật toán mã hóa trong điều kiện có lỗi

#### Mô tả kịch bản

Kịch bản thử nghiệm số 3 được xây dựng nhằm đánh giá độ tin cậy, khả năng chịu lỗi và khả năng đảm bảo toàn vẹn dữ liệu của các thuật toán mã hóa trong

điều kiện truyền dữ liệu có lỗi. Nút gửi sẽ đọc dữ liệu từ cảm biến DHT22, sau đó tiến hành mã hóa dữ liệu. Trước khi truyền dữ liệu qua nút gửi, hệ thống chủ động chèn lỗi ngẫu nhiên trực tiếp lên dữ liệu sau mã hóa. Tại nút nhận, dữ liệu sẽ được giải mã tương ứng với thuật toán được chỉ định. Sau khi giải mã, toàn bộ dữ liệu sẽ được so sánh với dữ liệu ban đầu nhằm đánh giá mức độ sai lệch dữ liệu sau truyền. Đối với thuật toán Ascon-128, quá trình mã hóa đồng thời tạo authentication tag để phục vụ việc xác thực tính toàn vẹn dữ liệu tại nút nhận. Nếu authentication tag không hợp lệ, gói dữ liệu sẽ bị loại bỏ khỏi quá trình xử lý.

### **Tiêu chí đánh giá**

Trong kịch bản này, các thuật toán được đánh giá thông qua các tiêu chí sau:

- Độ toàn vẹn dữ liệu: Tỷ lệ số gói dữ liệu được giải mã chính xác hoàn toàn trên tổng số gói dữ liệu được chấp nhận.
- Khả năng phát hiện lỗi: Khả năng phát hiện và loại bỏ các gói dữ liệu lỗi.
- Khả năng chấp nhận dữ liệu: Tỷ lệ số gói dữ liệu được chấp nhận xử lý trên tổng số gói dữ liệu được truyền.
- Số byte lỗi trung bình: Số byte trung bình bị sai lệch giữa dữ liệu gốc và dữ liệu sau giải mã trên các gói dữ liệu bị lỗi.
- Số bit lỗi trung bình: số bit trung bình bị sai lệch giữa dữ liệu gốc và dữ liệu sau giải mã trên các gói dữ liệu bị lỗi.

### **Cấu hình thử nghiệm**

- Kích thước gói dữ liệu: 32 bytes.
- Số lần truyền cho mỗi thuật toán: 200 lần.
- Tổng số gói dữ liệu: 600 gói.
- Xác suất chèn lỗi: 0.5%.
- Giao tiếp truyền thông: UART.
- Nền tảng phần cứng: ESP32.
- Tốc độ truyền: 115200 baudrate.

- Cả ba thuật toán AES, ChaCha20, Ascon được triển khai đồng thời trên cùng một hệ thống.

### **Phương pháp thử nghiệm và tính toán**

Hệ thống vận hành theo cơ chế lặp lại nhiều lần nhằm thu thập số liệu đánh giá tin cậy của từng thuật toán trong môi trường truyền có lỗi. Trong mỗi vòng lặp, dữ liệu từ cảm biến DHT22 được mã hóa trước khi thực hiện bước chèn lỗi ngẫu nhiên lên dữ liệu đã mã hóa.

Mô hình lỗi được xây dựng bằng cách lật ngẫu nhiên một bit trong mỗi byte của dữ liệu sau mã hóa theo cơ chế:

$$\text{Data}[i] \oplus = (1 \ll \text{random}(0.8)) \quad (3.8)$$

Trong quá trình này, hệ thống đồng thời ghi nhận trạng thái gói dữ liệu có bị tác động lỗi hay không thông qua biến tampered.

Sau khi nhận dữ liệu, nút nhận thực hiện giải mã bằng thuật toán tương ứng. Dữ liệu sau giải mã được so sánh trực tiếp với plaintext gốc nhằm xác định mức độ sai lệch dữ liệu sau quá trình truyền.

Đối với thuật toán Ascon, hệ thống sẽ thực hiện kiểm tra authentication tag nhằm xác định dữ liệu có bị thay đổi trong quá trình truyền hay không. Nếu tag không hợp lệ, dữ liệu sẽ bị từ chối và không được đưa vào xử lý tiếp theo. Cơ chế này giúp đảm bảo rằng các gói dữ liệu được chấp nhận đều là dữ liệu nguyên vẹn và chưa bị thay đổi.

Tỷ lệ phát hiện lỗi của Ascon được xác định theo công thức sau:

$$\text{Detection rate} = \frac{N_{\text{detected}}}{N_{\text{tampered}}} * 100\% \quad (3.9)$$

Tỷ lệ toàn vẹn dữ liệu được xác định theo công thức sau:

$$\text{Integrity} = \frac{N_{\text{correct}}}{N_{\text{accepted}}} * 100 \quad (3.10)$$

Tỷ lệ dữ liệu được chấp nhận được xác định theo công thức sau:

$$\text{Availability} = \frac{N_{\text{accepted}}}{N_{\text{total}}} * 100 \quad (3.11)$$

Số lỗi byte sai trung bình được xác định theo công thức sau:

$$\text{AvgByteError} = \frac{\text{Sum}_{\text{ByteError}}}{N_{\text{tampered}}} * 100 \quad (3.12)$$

Số lỗi bit sai trung bình được xác định theo công thức sau:

$$\text{AvgBitError} = \frac{\text{Sum}_{\text{BitError}}}{N_{\text{tampered}}} * 100 \quad (3.13)$$

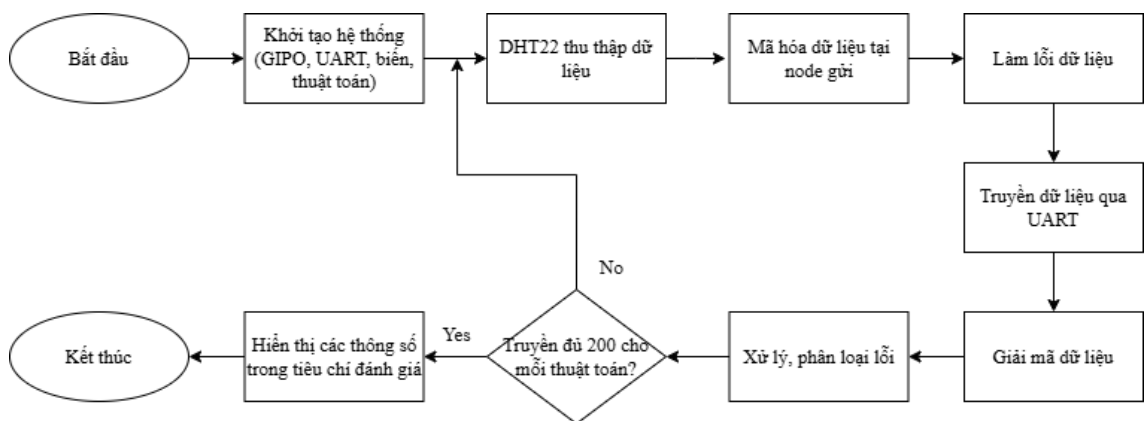
Trong đó:

- $N_{\text{total}}$ : Tổng số gói dữ liệu thử nghiệm.
- $N_{\text{detected}}$ : Số gói dữ liệu lỗi bị phát hiện.
- $N_{\text{tampered}}$ : Số gói dữ liệu lỗi được đánh dấu sau khi gây lỗi.
- $N_{\text{correct}}$ : Số gói dữ liệu giải mã hoàn toàn chính xác.
- $N_{\text{accepted}}$ : Số gói dữ liệu được chấp nhận.
- $\text{Sum}_{\text{ByteError}}$ : Tổng số byte lỗi.
- $\text{Sum}_{\text{BitError}}$ : Tổng số bit lỗi.

Các chỉ số trên cho phép đánh giá đồng thời khả năng chịu lỗi, mức độ lan truyền lỗi và khả năng bảo vệ tính toàn vẹn dữ liệu của từng thuật toán trong môi trường truyền có nhiễu.

### Lưu đồ giải thuật và quy trình thử nghiệm

Dưới đây là lưu đồ giải thuật của kịch bản 3: Đánh giá độ tin cậy và toàn vẹn dữ liệu của các thuật toán mã hóa trong điều kiện có lỗi



Hình 3.13 Lưu đồ giải thuật kịch bản 3

Hình 3.13 mô tả lưu đồ giải thuật và quy trình thử nghiệm của kịch bản 3: Đánh giá độ tin cậy và toàn vẹn dữ liệu của các thuật toán mã hóa trong điều kiện

có lỗi. Lưu đồ này thể hiện toàn bộ quá trình xử lý dữ liệu từ bước khởi tạo hệ thống, mã hóa, chèn lỗi, truyền dữ liệu, giải mã và đánh giá kết quả.

Ban đầu, hệ thống khởi tạo GPIO, giao tiếp UART, cảm biến DHT22 và các thuật toán AES, ChaCha20, Ascon. Sau đó, dữ liệu nhiệt độ và độ ẩm được đọc từ cảm biến để tạo dữ liệu đầu vào cho quá trình thử nghiệm.

Tại nút gửi, dữ liệu được mã hóa bằng thuật toán tương ứng. Sau bước mã hóa, hệ thống thực hiện chèn lỗi ngẫu nhiên trước khi truyền dữ liệu qua UART đến nút nhận.

Khi nhận đủ dữ liệu, nút nhận thực hiện giải mã và so sánh kết quả với plaintext gốc nhằm đánh giá mức độ sai lệch dữ liệu. Đối với Ascon, authentication tag đồng thời được kiểm tra để xác thực tính toàn vẹn dữ liệu.

Sau mỗi lần thử nghiệm, hệ thống kiểm tra điều kiện hoàn thành đủ 200 lần chạy cho từng thuật toán. Nếu chưa đạt, quy trình tiếp tục lặp lại; nếu hoàn tất, hệ thống tổng hợp và hiển thị các thông số đánh giá.

Lưu đồ trong Hình 3.13 cho thấy kịch bản thử nghiệm số 3 được xây dựng theo cơ chế tuần tự và lặp lại có kiểm soát, đảm bảo việc đánh giá khả năng chịu lỗi và bảo toàn dữ liệu của các thuật toán được thực hiện khách quan và nhất quán trong cùng điều kiện thử nghiệm

# CHƯƠNG 4

## KẾT QUẢ

### 4.1 MÔ HÌNH PHẦN CỨNG THỰC NGHIỆM

Hình bên dưới trình bày mô hình phần cứng thực nghiệm của hệ thống, bao gồm cảm biến DHT22, màn hình OLED SSD1306, hai vi điều khiển ESP32 đảm nhận vai trò nút gửi nút nhận:

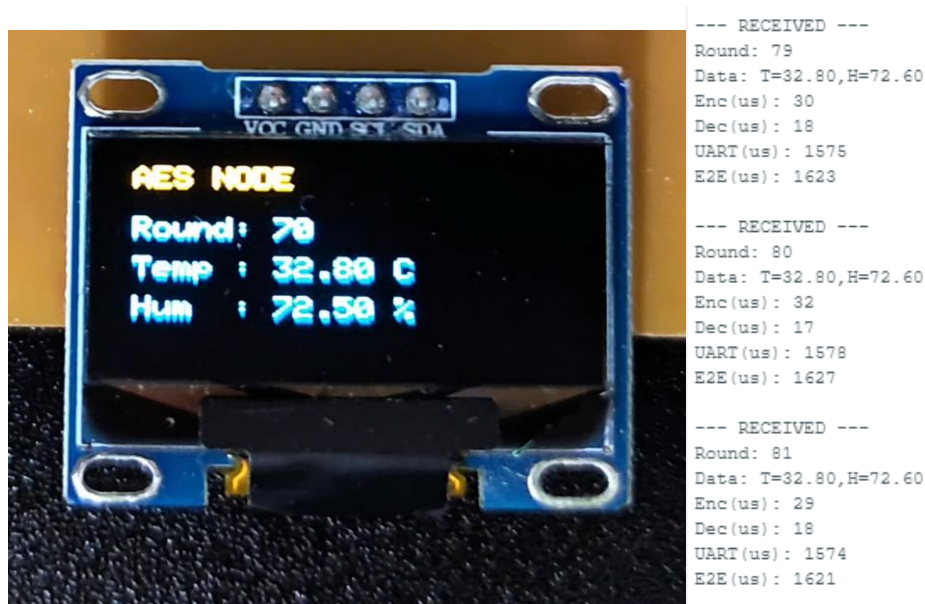


Hình 4.1 Mô hình phần cứng thực nghiệm của hệ thống

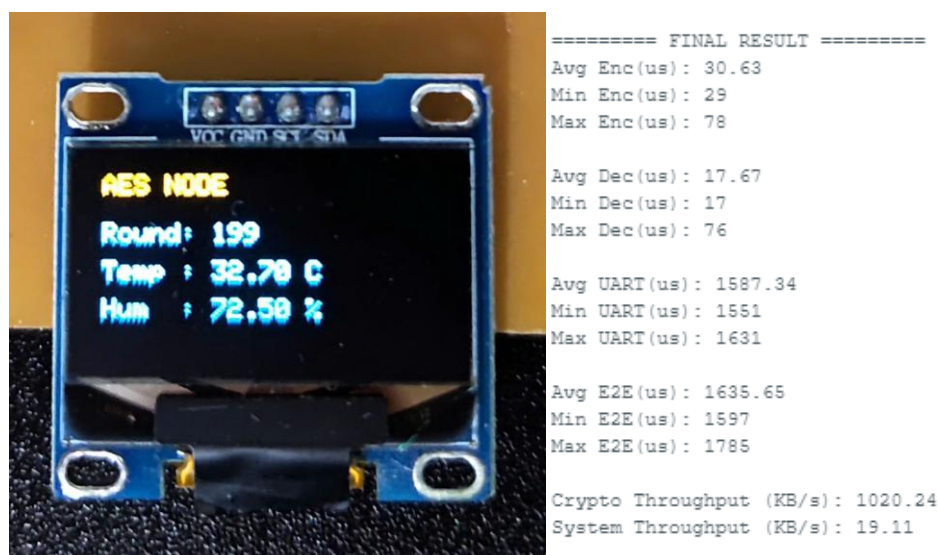
Hình 4.1 trình bày mô hình phần cứng thực nghiệm của hệ thống sử dụng hai vi điều khiển ESP32 giao tiếp với nhau thông qua UART. Trong mô hình, ESP32 phía gửi được kết nối với cảm biến DHT22 để thu thập dữ liệu nhiệt độ và độ ẩm, hiển thị thông tin lên màn hình OLED rồi truyền dữ liệu đến ESP32 phía nhận. ESP32 phía nhận sẽ xử lý thông tin, laptop sẽ đảm nhận việc hiển thị những thông tin ở ESP32 phía nhận đã xử lý trên màn hình serial monitor.

### 4.2 KẾT QUẢ THỰC THI HỆ THỐNG

Hình 4.2 và 4.3 trình bày quá trình và kết quả hoạt động của kịch bản truyền định kỳ, mỗi thuật toán đều có cấu trúc hiển thị tương tự:



Hình 4.2 Quá trình đo đặc trên phân cứng và Serial Monitor của kịch bản thử nghiệm số 1



Hình 4.3 Kết quả hoạt động của kịch bản thử nghiệm số 1

Hình 4.2 thể hiện quá trình hệ thống hoạt động, hiển thị thuật toán đang thử nghiệm, thông tin lần gửi cùng với dữ liệu được gửi. Serial mornito sẽ ghi nhận các thông số gồm thời gian mã hóa, giải mã, thời gian truyền UART, độ trễ toàn hệ thống và thông lượng ở mỗi lần thử nghiệm. Hình 4.3 là kết quả hiển thị sau 200 lần gửi dữ liệu và thông số thu được.

Hình 4.4 và hình 4.5 trình bày quá trình hoạt động của kịch bản thử nghiệm số 2 và kết quả cuối cùng của kịch bản, mỗi thuật toán đều có cấu trúc hiển thị tương tự:



Hình 4.4 Quá trình hoạt động của kịch bản thử nghiệm số 2



Hình 4.5 Kết quả hoạt động của kịch bản thử nghiệm số 2

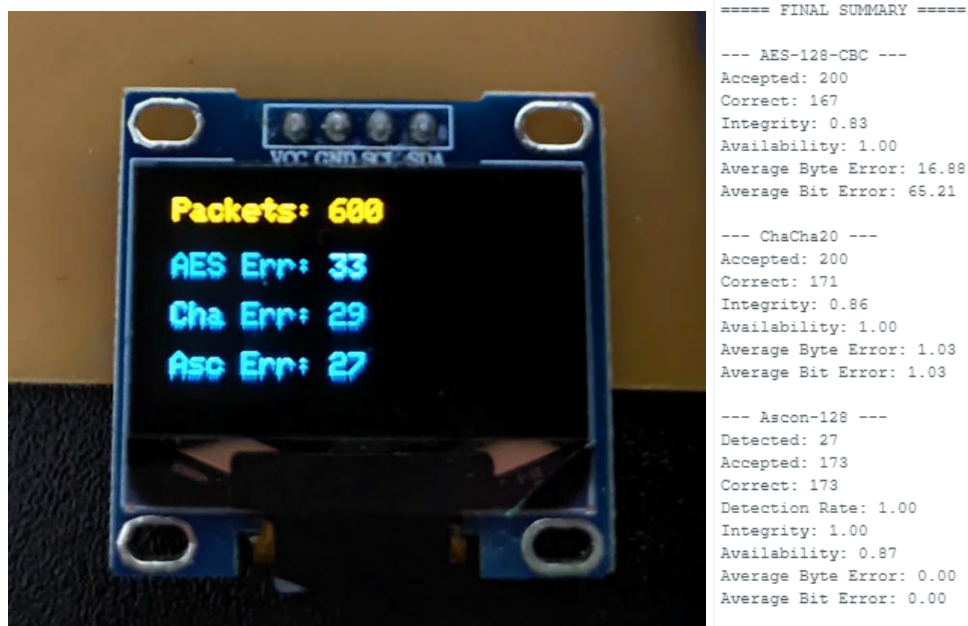
Hình 4.4 cho thấy trong quá trình hoạt động, màn hình OLED sẽ cập nhập số gói dữ liệu gửi liên tục từ nút gửi, trong lúc đó màn hình serial monitor của nút nhận sẽ không hiển thị gì, để tránh việc hiển thị làm cản trở tốc độ hoạt động của hệ thống ở phía nhận. Hình 4.4 cho thấy sau thời gian hoạt động 3 phút, màn hình OLED sẽ hiển thị tổng số gói gửi được, còn màn hình serial monitor sẽ hiển thị các thông số thu được.



Hình 4.6 và hình 4.7 trình bày quá trình hoạt động và kết quả hoạt động của kịch bản 3:



Hình 4.6 Quá trình hoạt động của kịch bản số 3



Hình 4.7 Kết quả hoạt động của kịch bản số 3

Hình 4.6 cho thấy trong quá trình hoạt động, màn hình OLED sẽ hiển thị quá trình hoạt động bên nút gửi, bao gồm số gói đang gửi đi cùng với số lượng gói dữ liệu bị lỗi của từng thuật toán. Hình 4.6 cho thấy sau khi gửi xong 600 gói dữ liệu, màn hình OLED sẽ hiển thị tổng số gói và số gói bị lỗi của mỗi thuật toán đã gửi đi. Màn hình serial monitor hiển thị các kết quả tiêu chí đánh giá thu được.

Các kết quả hiển thị trên Serial Monitor cho thấy hệ thống thực hiện thành công quá trình mã hóa, truyền nhận dữ liệu và thu thập các thông số phục vụ đánh giá hiệu năng cũng như độ tin cậy của các thuật toán trên nền tảng ESP32.

### 4.3 KẾT QUẢ ĐÁNH GIÁ THUẬT TOÁN

#### 4.3.1 Kịch bản thử nghiệm số 1: Nút cảm biến truyền dữ liệu định kỳ

Để đánh giá hiệu năng của các thuật toán trong kịch bản truyền định kỳ, hệ thống được cấu hình với số lần truyền là 200 lần, thời gian giữa các lần truyền là 2 giây, tốc độ baudrate là 115200 bps, kích thước gói dữ liệu 32 bytes. Mỗi thuật toán được triển khai trên một hệ thống độc lập.

Bảng 4.1 sẽ trình bày các thông số liên quan đến hiệu suất của thuật toán:

**Bảng 4.1** Hiệu năng hoạt động của các thuật toán của kịch bản số 1

| Thông số                                 | Thuật toán         |                 |                  |
|------------------------------------------|--------------------|-----------------|------------------|
|                                          | <i>AES-128-CBC</i> | <i>ChaCha20</i> | <i>Ascon-128</i> |
| <i>Thời gian mã hóa trung bình (us)</i>  | 31.64              | <b>26.11</b>    | 32.74            |
| <i>Thời gian mã hóa nhỏ nhất (us)</i>    | 31                 | 25              | 32               |
| <i>Thời gian mã hóa lớn nhất (us)</i>    | 79                 | 56              | 57               |
| <i>Thời gian giải mã trung bình (us)</i> | 17.83              | <b>12.16</b>    | 33.35            |
| <i>Thời gian giải mã nhỏ nhất (us)</i>   | 17                 | 11              | 33               |
| <i>Thời gian giải mã lớn nhất (us)</i>   | 76                 | 58              | 65               |
| <i>Thông lượng thuật toán (KB/s)</i>     | 987.83             | <b>1196.86</b>  | 954.64           |

Kết quả trong bảng 4.1 cho thấy trong ba thuật toán, ChaCha20 là thuật toán có thời gian mã hóa, giải mã và thông lượng thuật toán nhanh nhất. Điều này xuất

phát từ việc ChaCha20 sử dụng các phép toán đơn giản như XOR, cộng module và xoay bit, giúp tối ưu xử lý phần mềm trên ESP32.

Bảng 4.2 sẽ trình bày các thông số liên quan đến hiệu suất truyền dẫn của hệ thống:

**Bảng 4.2** Hiệu suất hoạt động của hệ thống của kịch bản số 1

| Thông số                                     | Thuật toán         |                 |                  |
|----------------------------------------------|--------------------|-----------------|------------------|
|                                              | <i>AES-128-CBC</i> | <i>ChaCha20</i> | <i>Ascon-128</i> |
| <i>Thời gian truyền UART trung bình (us)</i> | 1587.63            | <b>1587.8</b>   | 2320.81          |
| <i>Thời gian truyền UART nhỏ nhất (us)</i>   | 1554               | 1553            | 2284             |
| <i>Thời gian truyền UART lớn nhất (us)</i>   | 1636               | 1640            | 2388             |
| <i>Độ trễ hệ thống trung bình (us)</i>       | 1637.09            | <b>1626.06</b>  | 2386.9           |
| <i>Độ trễ hệ thống nhỏ nhất (us)</i>         | 1602               | 1591            | 2349             |
| <i>Độ trễ hệ thống lớn nhất (us)</i>         | 1791               | 1754            | 2510             |
| <i>Thông lượng hệ thống (KB/s)</i>           | 19.09              | 19.22           | <b>19.64</b>     |

Bảng 4.2 cho thấy sự chênh lệch giữa thời gian truyền của các thuật toán không quá lớn do bị giới hạn bởi tốc độ UART 115200 bps. Dù vậy, ChaCha20 vẫn đạt độ trễ hệ thống thấp nhất nhờ vào hiệu năng xử lý nhanh nhất. Trong khi đó, dù Ascon có độ trễ lớn nhất, thông lượng hệ thống của Ascon đạt giá trị cao nhất, do gói dữ liệu có kích thước lớn hơn do tag, nên làm tăng tổng lượng dữ liệu truyền mỗi giây.

Bảng 4.3 sẽ trình bày tài nguyên mà các thuật toán sử dụng:

**Bảng 4.3** Kết quả tài nguyên sử dụng của kịch bản số 1

| Thông số                 | Thuật toán         |                     |                    |
|--------------------------|--------------------|---------------------|--------------------|
|                          | <i>AES-128-CBC</i> | <i>ChaCha20</i>     | <i>Ascon-128</i>   |
| <i>Bộ nhớ RAM (KB)</i>   | Gửi: 24.27         | Gửi: 23.54          | <b>Gửi: 23.47</b>  |
|                          | Nhận: 25.12        | Nhận: 25.17         | <b>Nhận: 25.12</b> |
| <i>Bộ nhớ Flash (KB)</i> | Gửi: 324.29        | <b>Gửi: 322.85</b>  | Gửi: 323.93        |
|                          | Nhận: 284.36       | <b>Nhận: 282.88</b> | Nhận: 284.28       |

Bảng 4.3 cho thấy ChaCha20 sử dụng dung lượng Flash thấp nhất trong ba thuật toán được triển khai. Trong khi đó, Ascon có mức sử dụng RAM thấp nhất. Nhìn chung, ChaCha20 và Ascon đều cho thấy khả năng tối ưu tài nguyên tốt hơn AES khi triển khai trên ESP32, phù hợp với các hệ thống nhúng có tài nguyên hạn chế.

Từ ba bảng kết quả và phân tích trên, thuật toán ChaCha20 là thuật toán phù hợp nhất cho kịch bản truyền định kỳ trên hệ thống nhúng tài nguyên hạn chế nhờ vào hiệu năng cao, độ trễ thấp và mức sử dụng tài nguyên thấp.

#### 4.3.2 Kịch bản thử nghiệm số 2: Truyền dữ liệu liên tục

Để đánh giá hiệu năng của các thuật toán trong kịch bản truyền liên tục, hệ thống được cấu hình với thời gian truyền liên tục 3 phút cho mỗi thuật toán, tốc độ baudrate là 115200 bps, kích thước gói dữ liệu là 32 bytes. Mỗi thuật toán được triển khai trên một hệ thống độc lập để đảm bảo các thuật toán không ảnh hưởng đến hiệu suất của nhau.

Bảng 4.4 trình bày kết quả thông số liên quan đến hiệu năng của thuật toán

**Bảng 4.4** Hiệu năng hoạt động của các thuật toán của kịch bản số 2

| Thông số                                     | Thuật toán              |                 |                  |
|----------------------------------------------|-------------------------|-----------------|------------------|
|                                              | <i>AES-128-<br/>CBC</i> | <i>ChaCha20</i> | <i>Ascon-128</i> |
| <i>Thời gian mã hóa trung bình<br/>(us)</i>  | 17.32                   | <b>13.81</b>    | 35.35            |
| <i>Thời gian mã hóa nhỏ nhất (us)</i>        | 17                      | 13              | 32               |
| <i>Thời gian mã hóa lớn nhất (us)</i>        | 74                      | 55              | 60               |
| <i>Thời gian giải mã trung bình<br/>(us)</i> | 17.4                    | <b>11.86</b>    | 33.18            |
| <i>Thời gian giải mã nhỏ nhất (us)</i>       | 17                      | 11              | 33               |
| <i>Thời gian giải mã lớn nhất (us)</i>       | 76                      | 59              | 68               |
| <i>Thông lượng thuật toán (KB/s)</i>         | 1804.78                 | <b>2262.99</b>  | 883.98           |
| <i>Chi phí mã hóa (us/byte)</i>              | 0.54                    | <b>0.43</b>     | 1.10             |
| <i>Chi phí giải mã (us/byte)</i>             | 0.54                    | <b>0.37</b>     | 1.04             |

Kết quả bảng 4.4 cho thấy cả ba thuật toán đều duy trì được độ ổn định tương đối trong quá trình hoạt động liên tục. Giá trị trung bình đều gần với giá trị nhỏ nhất, cho thấy duy trì ở mức ổn định và gần với điều kiện hoạt động tối ưu ở mỗi lần xử lý dữ liệu. Một số giá trị cực đại ở thời gian giải mã xuất hiện chủ yếu do cơ chế ngắt UART hoặc xử lý buffer bất đồng bộ của ESP32, thay vì do bản thân thuật toán gây ra.

Về hiệu năng xử lý, ChaCha20 đạt thời gian mã hóa và giải mã thấp nhất, có thông lượng thuật toán cao nhất và chi phí xử lý thấp nhất trong ba thuật toán khảo sát. Kết quả này cho thấy trong điều kiện truyền dữ liệu liên tục, ChaCha20 vẫn duy trì được hiệu năng xử lý rất tốt trên nền tảng ESP32.

Bảng 4.5 sẽ trình bày các thông số liên quan đến hiệu suất truyền dẫn của hệ thống

**Bảng 4.2** Hiệu suất hoạt động của hệ thống của kịch bản số 2

| Thông số                                         | Thuật toán              |                 |                  |
|--------------------------------------------------|-------------------------|-----------------|------------------|
|                                                  | <i>AES-128-<br/>CBC</i> | <i>ChaCha20</i> | <i>Ascon-128</i> |
| <i>Tổng gói dữ liệu</i>                          | 27553                   | 27550           | 19385            |
| <i>Tổng dữ liệu (KB)</i>                         | 861.03                  | 860.94          | <b>908.67</b>    |
| <i>Thời gian truyền UART trung<br/>bình (us)</i> | 5855.13                 | 5860.97         | 8591.78          |
| <i>Thời gian truyền UART nhỏ<br/>nhất (us)</i>   | 1571                    | 1572            | 2288             |
| <i>Thời gian truyền UART lớn nhất<br/>(us)</i>   | 9186                    | 9185            | 9778             |
| <i>Độ trễ hệ thống trung bình (us)</i>           | 5889.84                 | <b>5886.64</b>  | 8660.32          |
| <i>Độ trễ hệ thống nhỏ nhất (us)</i>             | 1605                    | 1598            | 2359             |
| <i>Độ trễ hệ thống lớn nhất (us)</i>             | 9223                    | 9212            | 9849             |
| <i>Thông lượng hệ thống (KB/s)</i>               | 5.31                    | 5.31            | <b>5.41</b>      |
| <i>Thông lượng thực nghiệm<br/>(KB/s)</i>        | 4.78                    | 4.78            | <b>5.05</b>      |

Kết quả bảng 4.5 cho thấy, trong cùng khoảng thời gian thử nghiệm, dù Ascon có tổng gói xử lý dữ liệu thấp nhất trong ba thuật toán, nhưng tổng lượng dữ liệu truyền của Ascon lại cao hơn so với AES và ChaCha20. Nguyên nhân là do gói dữ liệu có tích hợp thêm authentication tag, làm kích thước mỗi gói dữ liệu lớn hơn AES và ChaCha20 khoảng 16 bytes.

Ngoài ra, các giá trị độ trễ và thời gian truyền của hệ thống nhìn chung vẫn duy trì ổn định trong suốt quá trình truyền liên tục, cho thấy hệ thống có khả năng hoạt động ổn định dưới điều kiện tải cao. Mặc dù ChaCha20 có hiệu năng xử lý nhanh nhất, Ascon vẫn đạt thông lượng hệ thống và thông lượng thực

nghiệm cao hơn nhờ lượng dữ liệu truyền trong mỗi gói lớn hơn. Do đó, giá trị thông lượng thực nghiệm của Ascon phản ánh tổng lượng dữ liệu truyền thành công qua hệ thống nhiều hơn là tốc độ xử lý riêng của thuật toán

Bảng 4.6 sẽ trình bày tài nguyên mà các thuật toán sử dụng:

**Bảng 4.6** Kết quả tài nguyên sử dụng của kịch bản số 2

| Thông số                 | Thuật toán         |                    |                    |
|--------------------------|--------------------|--------------------|--------------------|
|                          | <i>AES-128-CBC</i> | <i>ChaCha20</i>    | <i>Ascon-128</i>   |
| <i>Bộ nhớ RAM (KB)</i>   | Gửi: 23.58         | Gửi: 23.63         | <b>Gửi: 23.57</b>  |
|                          | Nhận: 22.05        | Nhận: 22.11        | <b>Nhận: 22.05</b> |
| <i>Bộ nhớ Flash (KB)</i> | Gửi: 322.95        | <b>Gửi: 321.44</b> | Gửi: 322.7         |
|                          | Nhận: 284.79       | <b>Nhận: 283.3</b> | Nhận: 284.84       |

Bảng 4.6 cho thấy ChaCha20 sử dụng dung lượng Flash thấp nhất trong ba thuật toán được triển khai. Trong khi đó, Ascon có mức sử dụng RAM thấp nhất. Nhìn chung, trong kịch bản truyền liên tục này, ChaCha20 và Ascon đều cho thấy khả năng tối ưu tài nguyên tốt hơn AES khi triển khai trên ESP32, phù hợp với các hệ thống nhúng có tài nguyên hạn chế.

Từ các bảng kết quả và phân tích trên, trong kịch bản truyền liên tục, ChaCha20 là thuật toán tối nhất nhờ hiệu năng cao và mức sử dụng tài nguyên thấp trên nền tảng ESP32. Trong khi đó, nếu hệ thống yêu cầu bảo mật cao thì Ascon là lựa chọn tối ưu nhờ cơ chế authentication tag và vẫn duy hiệu suất hệ thống hoạt động tốt trong suốt quá trình truyền liên tục.

### 4.3.3 Kịch bản thử nghiệm số 3: Đánh giá độ tin cậy và toàn vẹn dữ liệu của các thuật toán mã hóa trong điều kiện có lỗi

Hệ thống được cấu hình với 200 lần truyền dữ liệu cho mỗi thuật toán, tổng cộng 600 gói dữ liệu, tỉ lệ gây lỗi mỗi gói là 0.5%, tốc độ baudrate là 115200 bps, kích thước gói dữ liệu là 32 bytes. Cả ba thuật toán đều được triển khai đồng thời trên một hệ thống.

Dưới đây là bảng 4.7 sẽ trình bày kết quả đánh giá mức độ lỗi khi giải mã gói dữ liệu truyền có lỗi:

**Bảng 4.7** Mức độ lỗi của gói dữ liệu bị lỗi sau giải mã

| Thông số                         | Thuật toán         |                 |                  |
|----------------------------------|--------------------|-----------------|------------------|
|                                  | <i>AES-128-CBC</i> | <i>ChaCha20</i> | <i>Ascon-128</i> |
| <i>Số byte bị lỗi trung bình</i> | <b>16.94</b>       | 1.03            | 0                |
| <i>Số bit bị lỗi trung bình</i>  | <b>66.64</b>       | 1.03            | 0                |

Kết quả trong Bảng 4.7 cho thấy sự khác biệt rõ rệt giữa các thuật toán trong điều kiện truyền dữ liệu có lỗi. Khi xuất hiện lỗi trên ciphertext, khả năng giải mã chính xác của các thuật toán đều bị ảnh hưởng, tuy nhiên mức độ lan truyền lỗi và khả năng bảo vệ toàn vẹn dữ liệu là khác nhau.

Đối với AES-128-CBC, số byte và số bit lỗi trung bình cao nhất trong ba thuật toán. Điều này cho thấy lỗi trên ciphertext có xu hướng lan truyền mạnh khi sử dụng chế độ CBC, làm lỗi lan truyền trên nhiều byte của plaintext sau giải mã. Trong khi đó, ChaCha20 có số byte và số bit lỗi trung bình rất thấp, cho thấy lỗi chỉ ảnh hưởng cục bộ đến dữ liệu tương ứng thay vì lan rộng trên toàn bộ plaintext.

Dưới đây là bảng 4.8 sẽ trình bày kết quả đánh giá độ tin cậy và toàn vẹn của các gói dữ liệu:



**Bảng 4.8** Kết quả đánh giá độ tin cậy và toàn vẹn

| Thông số                                | Thuật toán              |                 |                  |
|-----------------------------------------|-------------------------|-----------------|------------------|
|                                         | <i>AES-128-<br/>CBC</i> | <i>ChaCha20</i> | <i>Ascon-128</i> |
| <i>Số gói giải mã chính xác</i>         | 167                     | 171             | 173              |
| <i>Số gói giải mã lỗi lỗi</i>           | 33                      | 29              | 27               |
| <i>Số gói được chấp nhận</i>            | 200                     | 200             | 173              |
| <i>Tỷ lệ phát hiện lỗi (%)</i>          | 0                       | 0               | <b>100</b>       |
| <i>Tỷ lệ toàn vẹn dữ liệu (%)</i>       | 83                      | 86              | <b>100</b>       |
| <i>Tỷ lệ dữ liệu được chấp nhận (%)</i> | 100                     | 100             | <b>87</b>        |

Khác với AES-128-CBC và ChaCha20, Ascon-128 đạt tỷ lệ phát hiện lỗi 100% nhờ cơ chế xác thực dữ liệu tích hợp (AEAD). Các gói dữ liệu bị thay đổi trong quá trình truyền đều bị phát hiện và loại bỏ trước khi giải mã, giúp tỷ lệ toàn vẹn dữ liệu đạt 100% và không xuất hiện lỗi byte hoặc lỗi bit trên các gói được chấp nhận. Tuy nhiên, do cơ chế loại bỏ dữ liệu lỗi, tỷ lệ dữ liệu được chấp nhận của Ascon-128 thấp hơn hai thuật toán còn lại.

Từ các kết quả có thể nhận thấy, trong các ứng dụng yêu cầu độ tin cậy và toàn vẹn dữ liệu cao như hệ thống IoT công nghiệp hoặc giám sát môi trường, các thuật toán có tích hợp cơ chế xác thực như Ascon là lựa chọn phù hợp nhất, đặc biệt trong việc đảm bảo tính chính xác hơn là việc duy trì toàn bộ dữ liệu.

# CHƯƠNG 5

## KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

### 5.1 KẾT LUẬN

Đề tài đã thực thi các thuật toán mã hóa AES, ChaCha20, Ascon trên nền tảng vi điều khiển ESP32 dưới các dạng kịch bản kiểm thử nút cảm biến truyền dữ liệu định kỳ, kịch bản truyền liên tục, kịch bản đánh giá độ tin cậy và toàn vẹn dữ liệu của thuật toán trong điều kiện có lỗi. Đề tài đã thu thập các thông số như thời gian xử lý, thông lượng, tài nguyên hệ thống cùng với khả năng phát hiện lỗi trên vi điều khiển thực ESP32 NodeMCU.

Sau qua trình nghiên cứu, triển khai và đánh giá thực nghiệm các thuật toán mã hóa AES, ChaCha20 và Ascon trên vi điều khiển ESP32, đề án rút ra được những kết luận như sau:

- Ở kịch bản truyền định kỳ, thuật toán ChaCha20 là thuật toán có hiệu năng hoạt động tốt nhất, với thông lượng thuật toán là 1196.86 KB/s, nhanh hơn 17.46% so với thuật toán AES (đạt 987.83 KB/s) và 20.24% so với thuật toán Ascon (đạt 954.64 KB/s). Tuy nhiên, ở hiệu suất hệ thống, thuật toán Ascon có thông lượng hệ thống là 19.64 KB/s, cao hơn 3.05% so với thuật toán AES (đạt 19.09 KB/s) và 2.14% so với thuật toán ChaCha20 (đạt 19.22 KB/s). Vì Ascon làm tăng gói dữ liệu thực tế trên đường bằng cách chèn thêm dữ liệu xác thực tag, khiến cho tổng dữ liệu xử lý được nhiều hơn chứ không phải tốc độ xử lý tăng hơn. Về tài nguyên sử dụng, Ascon chiếm tổng dung lượng RAM là 48.59KB, ít hơn 1.62% so với thuật toán AES (đạt 49.39 KB) và 0.25% so với thuật toán ChaCha20 (đạt 48.71 KB). Về dung lượng Flash, ChaCha20 chiếm tổng dung lượng là 605.73 KB, ít hơn 0.48% so với thuật toán AES (đạt 608.65 KB) và 0.41% so với thuật toán Ascon (đạt 608.21 KB).

- Ở kịch bản truyền liên tục, thuật toán ChaCha20 cũng là thuật toán có hiệu năng hoạt động tốt nhất, với thông lượng thuật toán là 2262.99 KB/s, nhanh hơn 20.25% so với thuật toán AES (đạt 1804.78 KB/s) và 60.94% so với thuật toán Ascon (đạt 883.98 KB/s). Tuy nhiên, ở hiệu suất hệ thống, thuật toán Ascon vẫn tiếp tục có thông lượng hệ thống là 5.41 KB/s, cao hơn 1.85% so với cả hai thuật toán AES và ChaCha20 (đều đạt 5.31 KB/s). Tương tự như kết quả ở kịch bản truyền định kì, kết quả phản ánh được ảnh hưởng của cơ chế chèn thêm dữ liệu xác thực tag của Ascon, đẩy tổng dữ liệu xử lý cao hơn chứ không phải tốc độ xử lý nhanh hơn. Về tài nguyên sử dụng, Ascon chiếm tổng dung lượng RAM là 45.62KB, ít hơn 0.02% so với thuật toán AES (đạt 45.63 KB) và 0.26% so với thuật toán ChaCha20 (đạt 45.74 KB). Về dung lượng Flash, ChaCha20 chiếm tổng dung lượng là 604.74 KB, ít hơn 0.48% so với thuật toán AES (đạt 607.64 KB) và 0.46% so với thuật toán Ascon (đạt 607.54 KB).
- Ở kịch bản truyền trong điều kiện có lỗi, thuật toán Ascon chiếm ưu thế tuyệt đối về khả năng phát hiện lỗi đạt 100%, hơn hoàn toàn khi thuật toán AES và ChaCha20 không có khả năng phát hiện lỗi (0%). Nguyên nhân là vì cấu trúc thuật toán thuần túy của AES và ChaCha20 không có khả năng phát hiện lỗi, nên dẫn đến tỉ lệ phát hiện lỗi là 0%.

Từ các kết luận trên, thuật toán ChaCha20 là thuật toán có hiệu năng hoạt động tốt nhất trong các hệ thống truyền định kỳ và truyền liên tục, tài nguyên sử dụng cũng được tối ưu. Tuy nhiên, dù hiệu năng hoạt động của thuật toán Ascon không lý tưởng, nhưng thuật toán Ascon vẫn tối ưu tài nguyên tối, có độ tin cậy và toàn vẹn cao nhất, phù hợp với các hệ thống yêu cầu bảo mật và độ tin cậy cao. Thuật toán AES mặc dù không nổi bật về hiệu năng hay khả năng xác thực dữ liệu, thuật toán vẫn duy trì khả năng hoạt động ổn định và có ưu thế về tính phổ biến cũng như khả năng tương thích trong nhiều hệ thống hiện nay.

## 5.2 HƯỚNG PHÁT TRIỂN

Mặc dù đã đạt được các mục tiêu đề ra, đề tài vẫn còn một số hướng có thể tiếp tục phát triển trong tương lai.

Trước hết, hệ thống hiện tại sử dụng giao tiếp UART để truyền dữ liệu giữa hai node ESP32. Trong các nghiên cứu tiếp theo, có thể mở rộng sang các giao thức truyền thông phổ biến trong IoT như WiFi, BLE hoặc MQTT nhằm đánh giá thuật toán trong môi trường truyền thông thực tế hơn.

Bên cạnh đó, đề tài hiện mới tập trung vào hiệu năng xử lý và độ tin cậy dữ liệu, chưa đánh giá mức tiêu thụ năng lượng của từng thuật toán. Đây là một yếu tố quan trọng đối với các thiết bị IoT sử dụng pin và cần được xem xét trong các nghiên cứu tiếp theo.

Ngoài ra, có thể mở rộng thực nghiệm trên các nền tảng phần cứng khác như STM32, ESP32-S3 hoặc Raspberry Pi Pico để đánh giá khả năng mở rộng và tính tương thích của các thuật toán trên nhiều kiến trúc hệ thống nhúng khác nhau.

Trong tương lai, hệ thống cũng có thể được mở rộng với nhiều kích thước dữ liệu và điều kiện truyền khác nhau nhằm đánh giá toàn diện hơn khả năng ứng dụng của các thuật toán mã hóa trong thực tế.

## PHỤ LỤC

### PHỤ LỤC A: MÃ NGUỒN CHƯƠNG TRÌNH

<https://github.com/khanhlthcm-hue/Final-Project-Code.git>

### PHỤ LỤC B: HƯỚNG DẪN SỬ DỤNG SẢN PHẨM THI CÔNG

Để kiểm tra, thu thập số liệu tính toán của một thuật toán, ta cần làm những việc sau:

- Lựa chọn kịch bản muốn chạy để nạp vào vi điều khiển. demo1, demo2 và demo3 sẽ tương ứng với kịch bản 1 truyền định kỳ, kịch bản 2 truyền liên tục, kịch bản 3 truyền trong điều kiện có lỗi.
- Chương trình có tên chứa chữ sender sẽ được nạp vào vi điều khiển ESP32 nút gửi, nằm bên trái của phần cứng. Chương trình có tên chứa chữ receive sẽ được nạp vào vi điều khiển ESP32 nút nhận, nằm bên phải phần cứng. Khi nạp, phải đảm bảo là hai chương trình đều cùng xài chung 1 thuật toán và cùng kịch bản hoạt động.
- Để quan sát kết quả, cắm dây cáp ở vi điều khiển ESP32 phía nhận để giám sát màn hình Serial Monitor.

## TÀI LIỆU THAM KHẢO

- [1] National Institute of Standards and Technology (NIST), *Advanced Encryption Standard (AES)*, FIPS PUB 197, Nov. 2001.
- [2] National Institute of Standards and Technology (NIST), *Recommendation for Block Cipher Modes of Operation*, SP 800-38A, Dec. 2001.
- [3] Y. Nir and A. Langley, *ChaCha20 and Poly1305 for IETF Protocols*, RFC 8439, Jun. 2018.
- [4] M. S. Turan, K. A. McKay, D. Chang, J. Kang, and J. Kelsey, *Ascon-Based Lightweight Cryptography Standards for Constrained Devices*, NIST Special Publication 800-232, Aug. 2025.
- [5] C. T. N. Quynh, N. T. Ngoc, L. T. Ninh, and P. T. T. An, "Throughput Optimization of the ASCON Lightweight Cryptographic Algorithm on IoT Devices," *J. Sci. Technol. Inf. Security*, vol. 3, no. 26, 2025.
- [6] Espressif Systems, *ESP32 Series Datasheet*, Version 5.2, 2023.
- [7] Aosong Electronics, *DHT22 (AM2302) Temperature and Humidity Sensor Datasheet*.
- [8] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schl  ffer, *Ascon v1.2: Submission to the CAESAR Competition*, 2016.
- [9] Sensirion AG, *Datasheet SHT3x-DIS Humidity and Temperature Sensor*, Version 3, Aug. 2016.
- [10] Bosch Sensortec, *BME280 Data Sheet*, Rev. 1.6, Sep. 2018.
- [11] M. A. Mazidi, J. G. Mazidi, and R. D. McKinlay, *The 8051 Microcontroller and Embedded Systems*. Upper Saddle River, NJ, USA: Pearson, 2006.
- [12] MS\_IT. *An toàn bảo mật thông tin – Chương 3 Mã hóa AES*. (2020). Accessed: Feb. 23, 2026. [Online Video]. Available: <https://youtu.be/DUSmxMyHF-g>

- [13] Computerphile. *Chacha Cipher – Computerphile*. (2021). Accessed: Feb. 23, 2026. [Online Video]. Available: <https://youtu.be/UeIpq-C-GSA>
- [14] C. Tezcan. *ASCON Authenticated Encryption Scheme*. (2020). Accessed: Feb. 23, 2026. [Online Video]. Available: <https://youtu.be/YIjiTB9ZxYw>
- [15] Solomon Systech, *SSD1306: 128 × 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller*, Rev. 1.1, Apr. 2008.
- [16] Circuit Basics. (2023). *UART Communication Protocol* [Online]. Available: <https://www.circuitbasics.com/basics-uart-communication/>. Accessed: Jun. 8, 2026.
- [17] National Institute of Standards and Technology (NIST). (2018, Sep.). *NIST's Encryption Standard Has Minimum \$250 Billion Economic Benefit* [Online]. Available: <https://www.nist.gov/news-events/news/2018/09/nists-encryption-standard-has-minimum-250-billion-economic-benefit>. Accessed: Jun. 8, 2026.
- [18] National Institute of Standards and Technology (NIST). *Cryptography* [Online]. Available: <https://csrc.nist.gov/nist-cyber-history/cryptography/chapter>. Accessed: Jun. 8, 2026.
- [19] J.-P. Aumasson, *Serious Cryptography: A Practical Introduction to Modern Encryption*. San Francisco, CA, USA: No Starch Press, 2017.
- [20] RWeather. *Crypto Library for Arduino*. [Online]. Available: <https://github.com/rweather/arduinolibs>. Accessed: Jun. 8, 2026.
- [21] The Gioi IC. *Linh kiện điện tử và module nhúng* [Online]. Available: <https://www.thegioic.com/>. Accessed: Jun. 8, 2026.
- [22] Arduino. *Arduino Uno Rev3 Technical Specifications* [Online]. Available: <https://docs.arduino.cc/hardware/uno-rev3>. Accessed: Jun. 8, 2026.
- [23] Espressif Systems, *ESP8266EX Datasheet*, Version 7.0, 2023.
- [24] ILI Technology Corp., *ILI9341: a-Si TFT LCD Single Chip Driver 240RGB × 320 Resolution and 262K Color*, Version 1.11, 2011.

- [25] DFRobot, *LCD1602 RGB Backlight Module Datasheet*, 2013.
- [26] R. Mischianti, “ESP32 NODEMCU-32S ESP-32S Kit: high resolution pinout, datasheet, and specs,” *Mischianti.org*, Feb. 1, 2024. [Online]. Available: [Mischianti.org](https://mischianti.org). [Accessed: Jun. 11, 2026].